

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

1. Project Definition

Introduction

A **project** is a planned set of activities carried out to achieve a specific objective within a defined **scope, time, cost, and quality**.

In medical device development, a project may involve designing, developing, testing, validating, obtaining regulatory clearance/approval, and launching a new medical device.

Example

Development of an **IoT-based patient monitoring system** can be considered a project with:

- Objective – Develop a real-time patient monitoring system
- Duration – 12 months
- Budget – ₹20 lakh
- Team – Biomedical engineers, software engineers, clinicians and marketing personnel
- Deliverable – Tested and validated prototype/product

Major elements of project definition

1. **Project title**
2. **Problem statement**
3. **Objectives**
4. **Scope**
5. **Deliverables**
6. **Target customers**
7. **Timeline**
8. **Budget**
9. **Resources**
10. **Quality requirements**
11. **Regulatory requirements**
12. **Risk management**

Importance

A clear project definition helps the team understand **what has to be developed, why it is required, who will use it, how much it will cost, and when it should be completed**.

MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

2. Financial Targets

Financial targets are the **specific financial goals established for a product or project**.

They help determine whether the product is commercially viable.

Important financial targets

1. Development budget

Maximum amount allocated for product development.

2. Manufacturing cost

Cost required to manufacture one unit.

3. Selling price

Price at which the product is sold.

4. Revenue target

Expected income from product sales.

5. Profit target

Expected profit from the product.

6. Break-even point

The sales level at which total revenue equals total cost.

7. Return on Investment (ROI)

Measures the return generated from an investment.

$$\text{ROI} = (\text{Net Return} / \text{Investment}) \times 100$$

Example

Suppose:

- Manufacturing cost = ₹8,000/device
- Selling price = ₹15,000/device
- Expected sales = 1,000 devices

The company can estimate its revenue, contribution margin, operating costs, and expected profit before deciding whether to proceed with the project.

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

3. Physician Input

Introduction

Physician input refers to obtaining information, suggestions, feedback, and clinical requirements from doctors during medical product development.

Physicians are important stakeholders because they understand the **clinical problem, patient requirements, workflow, treatment procedures, and limitations of existing technologies.**

Importance of Physician Input

1. Identification of clinical problems

Physicians can identify problems that require technological solutions.

Example:

A physician may identify that existing patient monitoring systems provide information too late for early intervention.

2. Defining product requirements

Doctors can specify:

- Required parameters
- Accuracy
- Response time
- Clinical functionality
- Patient population
- Desired alerts

3. Product design

Physician feedback helps engineers develop a device suitable for actual clinical environments.

4. Usability

Doctors can evaluate:

- User interface
- Display
- Controls
- Workflow
- Ease of operation

5. Clinical validation

Physicians may participate in clinical evaluation and provide feedback regarding clinical performance.

6. Risk identification

Doctors can identify possible clinical hazards and inappropriate uses.

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

7. Product acceptance

A product developed with appropriate clinical input is more likely to be accepted by healthcare professionals.

Methods of Obtaining Physician Input

- Interviews
- Questionnaires
- Focus groups
- Clinical observations
- Product demonstrations
- Advisory panels
- Prototype evaluation
- Clinical studies

Example

For a **smart prosthetic limb**, engineers may develop the sensors and control system, while physicians and rehabilitation professionals can provide input regarding:

- Patient comfort
- Gait requirements
- Safety
- Socket fit
- Rehabilitation needs
- Clinical usability

4. FDA Approval

Introduction

The **U.S. Food and Drug Administration (FDA)** regulates medical devices marketed in the United States. FDA's Center for Devices and Radiological Health (CDRH) regulates firms that manufacture, repack, relabel, or import medical devices for the U.S. market.

An important point for students is that **not every medical device receives an "FDA approval."** Depending on the device and regulatory pathway, the product may be **510(k) cleared, De Novo classified, PMA approved, or exempt from a premarket submission.**

Objectives of FDA Regulation

FDA regulation aims to ensure that medical devices:

- Are reasonably safe
- Are effective for their intended use

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Meet applicable regulatory requirements
- Are appropriately manufactured
- Have appropriate labeling
- Continue to be monitored after marketing

Step 1 – Identify the Intended Use

The first step is to clearly define:

- What the device does
- Who will use it
- Which patients will use it
- What disease/condition it addresses
- Whether it diagnoses, treats, prevents, monitors, or supports a condition

The intended use and indications for use are important because they influence the device's classification and regulatory pathway.

Step 2 – Determine Device Classification

FDA generally classifies medical devices into:

Class I – Low risk

Generally subject to **General Controls**.

Examples may include certain low-risk devices.

Class II – Moderate risk

Subject to **General Controls + Special Controls**.

Class III – High risk

Subject to **General Controls + Premarket Approval (PMA)** in applicable cases.

FDA states that classification is risk-based, with Class I generally representing the lowest risk and Class III the highest.

Class	Risk	Main Controls
Class I	Low	General Controls
Class II	Moderate	General + Special Controls
Class III	High	General Controls + PMA in applicable cases

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Step 3 – Identify the Appropriate Regulatory Pathway

The major pathways students should know are:

A. 510(k) – Premarket Notification

The manufacturer generally demonstrates that the new device is **substantially equivalent** to a legally marketed predicate device.

The comparison generally considers:

- Intended use
- Technological characteristics
- Performance
- Safety

FDA explains that a 510(k) is a pathway based on substantial equivalence to an appropriate legally marketed device.

Important: The correct term is generally "**FDA clearance**", not "FDA approval," for a 510(k).

B. De Novo Classification

The **De Novo pathway** can be used for certain novel devices for which there is no suitable predicate but whose risks can be appropriately controlled.

A successful De Novo request establishes a new classification for that device type. FDA identifies De Novo as a pathway for certain novel device types that have not previously been marketed in the United States.

C. Premarket Approval – PMA

PMA is the most stringent major premarket pathway.

It is generally associated with **Class III devices** requiring PMA.

The manufacturer must provide scientific evidence supporting the safety and effectiveness of the device. FDA describes PMA as the scientific and regulatory review used to evaluate the safety and effectiveness of applicable Class III devices.

Examples of devices that may require PMA include certain implantable and life-supporting devices.

D. HDE

Humanitarian Device Exemption (HDE) is associated with certain devices intended to benefit patients with rare diseases or conditions meeting the applicable statutory criteria.

Step 4 – Preclinical/Nonclinical Testing

Before clinical use or marketing, applicable devices may require testing such as:

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Mechanical testing
- Electrical safety testing
- Biocompatibility
- Software verification
- Electromagnetic compatibility
- Sterilization validation
- Packaging testing
- Performance testing
- Animal testing where appropriate

The exact testing depends on the device.

Step 5 – Clinical Evaluation/Investigation

For devices requiring clinical evidence, clinical investigations may be conducted to evaluate:

- Safety
- Effectiveness
- Performance
- Patient outcomes
- Adverse events

Where applicable, an **Investigational Device Exemption (IDE)** permits certain investigational devices to be used in clinical studies in accordance with FDA requirements.

Step 6 – Quality Management System

Manufacturers must establish an appropriate quality system.

A major current development is that FDA's **Quality Management System Regulation (QMSR)** became effective on **February 2, 2026**. QMSR amended 21 CFR Part 820 and incorporates by reference **ISO 13485:2016** for medical device quality management systems.

Therefore, students should now understand:

21 CFR Part 820 → QMSR → incorporates ISO 13485:2016

The QMS supports areas such as:

- Design and development
- Manufacturing
- Quality control
- Documentation

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Corrective and preventive actions
- Complaint handling
- Risk management
- Records

Step 7 – Prepare Regulatory Submission

The manufacturer prepares the appropriate regulatory submission.

Depending on the pathway, this may include:

- Device description
- Intended use
- Indications for use
- Design information
- Risk information
- Performance data
- Test results
- Clinical data where required
- Manufacturing information
- Quality-system information
- Labeling

Step 8 – Labeling

Medical device labeling is an important regulatory requirement.

FDA's general device labeling requirements are contained in **21 CFR Part 801**.

Labeling may include:

- Device name
- Manufacturer information
- Intended use
- Indications
- Instructions for use
- Warnings
- Precautions
- Contraindications

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Installation instructions
- Maintenance information

Labeling must not provide misleading information or unsupported claims.

Step 9 – FDA Review

FDA reviews the submitted information according to the applicable pathway.

The review may involve:

- Technical evaluation
- Clinical evaluation
- Statistical evaluation
- Manufacturing/quality considerations
- Labeling review
- Risk assessment

FDA may request additional information where necessary.

Step 10 – Regulatory Decision

Depending on the pathway, the outcome may be:

- 510(k) clearance
- De Novo classification
- PMA approval
- HDE approval
- Exemption where applicable

Step 11 – Post-Market Requirements

FDA oversight does not end when a device reaches the market.

Post-market activities may include:

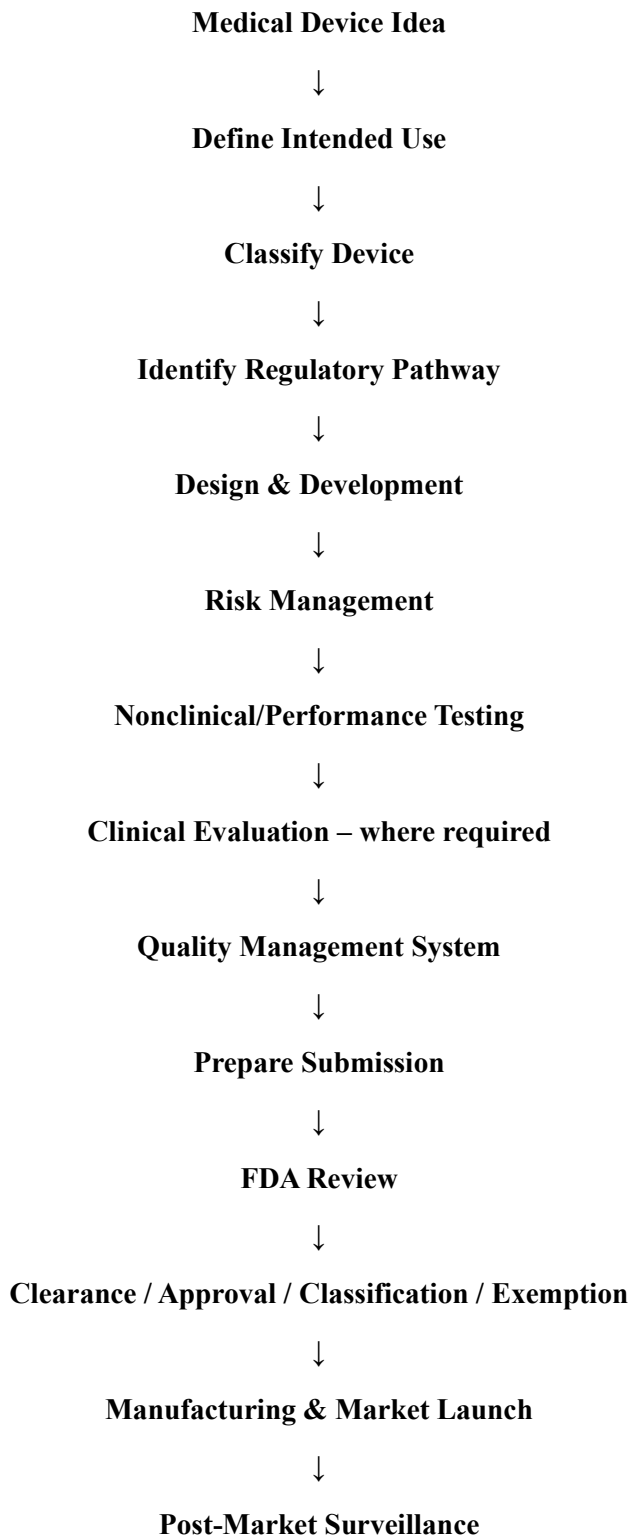
- Complaint handling
- Medical device reporting
- Corrective actions
- Recalls where necessary
- Post-market surveillance
- Manufacturing quality monitoring

MEDICAL DEVICE MARKETING UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Monitoring of device safety and performance

FDA states that it is responsible for assuring that medical devices available in the United States remain safe and effective throughout their total product lifecycle.

Simplified FDA Pathway



MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Important Difference: Clearance vs Approval

Term	Common pathway	Meaning
FDA Clearance	510(k)	Device has been determined substantially equivalent to an appropriate predicate
FDA Approval	PMA	FDA has approved the PMA based on applicable evidence of safety and effectiveness
De Novo Classification	De Novo	New device type classified through the De Novo pathway
Exempt	Certain devices	No applicable premarket submission, subject to the conditions and limitations of the exemption

5. Reimbursement

Introduction

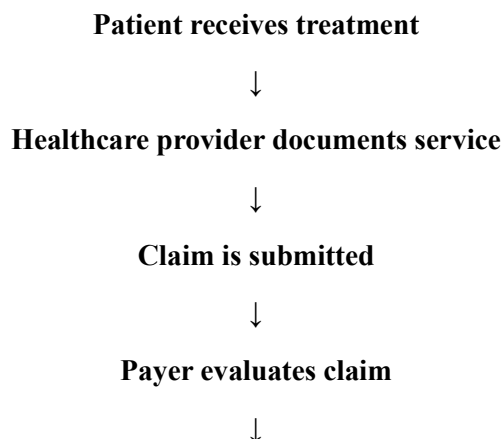
Reimbursement refers to the payment made to healthcare providers or organizations for eligible healthcare services, procedures, or products.

For a medical device company, reimbursement is important because a product may be clinically effective but commercially unsuccessful if customers cannot obtain adequate financial coverage for its use.

Major Participants

- Patient
- Doctor
- Hospital
- Insurance company/payer
- Government
- Medical device manufacturer

Basic Process



MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Claim approved/rejected



Payment made

Importance of Reimbursement

Reimbursement influences:

- Patient affordability
- Hospital purchasing
- Product adoption
- Market size
- Revenue
- Product pricing

Example

A new cardiac monitoring device may provide excellent clinical benefits. However, if hospitals or patients cannot recover the cost through an applicable payment mechanism, adoption may remain low.

6. Managing Issues

During product development and commercialization, several issues can arise.

Common issues

- Technical failures
- Component shortages
- Design changes
- Budget overruns
- Regulatory delays
- Testing failures
- Customer complaints
- Manufacturing problems
- Software errors
- Supplier delays

Issue Management Process

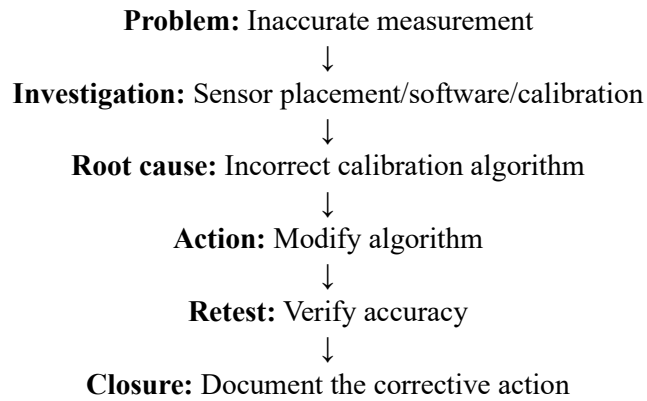
Identify issue → Analyze cause → Determine impact → Prioritize → Develop corrective action → Assign responsibility → Implement action → Monitor → Close issue

Example

If a prototype shows inaccurate SpO₂ readings:

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH



7. Meeting Deadlines

Meeting deadlines means completing project activities within the planned time.

Methods

1. Develop a project schedule

Define:

- Activities
- Start date
- End date
- Dependencies
- Responsible person

2. Prioritize critical activities

Identify activities that directly affect the final delivery date.

3. Monitor progress

Use:

- Gantt charts
- Milestone tracking
- Weekly meetings
- Progress reports

4. Allocate resources properly

Ensure availability of:

- Personnel
- Equipment
- Materials

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Budget

5. Manage risks

Identify potential delays before they occur.

6. Take corrective action

If an activity is delayed:

- Reallocate resources
- Change sequence
- Add manpower
- Revise schedule
- Address technical problems

Example

If clinical testing is delayed by two weeks, the team should assess whether documentation, software development, or manufacturing activities can proceed in parallel without compromising quality.

8. Product Launch

Introduction

Product launch is the process of introducing a new product into the target market and making it available to customers.

For a medical device, product launch is more than simply selling the product. It involves:

- Regulatory readiness
- Manufacturing readiness
- Market preparation
- Pricing
- Distribution
- Sales training
- Customer support
- Product communication
- Post-market monitoring

Stages of Product Launch

1. Market Preparation

Before launching the product, the company should understand:

- Target customers

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Market size
- Competitors
- Customer requirements
- Pricing
- Distribution channels
- Market barriers

2. Regulatory Readiness

The company must ensure that applicable regulatory requirements have been satisfied before marketing the device in the relevant jurisdiction.

This includes appropriate:

- Regulatory status
- Labeling
- Documentation
- Quality systems
- Manufacturing controls

3. Manufacturing Readiness

The company should ensure that sufficient quantities can be produced with consistent quality.

Activities include:

- Supplier qualification
- Production planning
- Quality inspection
- Packaging
- Sterilization where required
- Inventory planning

4. Pricing Strategy

The company determines the price based on:

- Manufacturing cost
- Competitor prices
- Customer willingness to pay
- Clinical value

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Distribution cost
- Market positioning
- Reimbursement environment

5. Sales Preparation

The sales team should understand:

- Product features
- Clinical benefits
- Product limitations
- Target customers
- Competitors
- Pricing
- Demonstration procedure
- Customer objections

6. Marketing and Promotion

Promotion may include:

- Product demonstrations
- Conferences
- Medical exhibitions
- Digital marketing
- Scientific publications
- Healthcare professional education
- Sales presentations

Medical device claims should remain consistent with applicable regulatory requirements and authorized/intended uses.

7. Distribution

The company determines how the product will reach customers.

Possible channels include:

Manufacturer → Distributor → Hospital → Patient

or

Manufacturer → Hospital

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

or

Manufacturer → Clinic → Patient

8. Product Training

Healthcare professionals may need training on:

- Installation
- Operation
- Maintenance
- Safety
- Troubleshooting
- Clinical use

9. Launch

The product is officially introduced to the target market.

Launch activities may include:

- Product announcement
- Demonstrations
- Distributor activation
- Sales campaigns
- Hospital visits
- Customer training

10. Post-Launch Monitoring

After launch, the company monitors:

- Sales
- Customer feedback
- Complaints
- Device failures
- Returns
- Adverse events
- Market acceptance
- Profitability

MEDICAL DEVICE MARKETING UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Product Launch Framework



Factors Determining Successful Product Launch

Technical readiness

The device should perform according to its specifications.

Clinical readiness

Healthcare professionals should understand its clinical value.

Regulatory readiness

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Required regulatory requirements must be satisfied.

Financial readiness

The company should have adequate financial resources.

Manufacturing readiness

Production should meet expected demand.

Sales readiness

Sales personnel should be properly trained.

Customer support

Installation, training, maintenance, and troubleshooting should be available.

Market readiness

There should be sufficient customer demand.

9. Forecasting

Forecasting is the process of **estimating future demand, sales, revenue, or market requirements**.

It helps companies determine how many products they should manufacture and how much inventory they should maintain.

Types of Forecasting

1. Qualitative forecasting

Based on expert opinion and market knowledge.

Methods include:

- Expert opinion
- Sales-force opinion
- Customer surveys
- Delphi method

2. Quantitative forecasting

Uses historical data and mathematical/statistical techniques.

Examples:

- Moving averages
- Trend analysis
- Regression analysis
- Time-series analysis

Factors Affecting Medical Device Forecasting

- Number of hospitals

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Patient population
- Disease prevalence
- Competitor products
- Product price
- Healthcare policies
- Reimbursement
- Technology adoption
- Economic conditions

Example

If a company sold:

- 500 units in Year 1
- 700 units in Year 2
- 900 units in Year 3

The company may use historical trends and market growth assumptions to estimate future demand.

Importance

Forecasting helps with:

- Production planning
- Inventory control
- Manpower planning
- Financial planning
- Sales targets
- Supply-chain management

10. Pricing

Pricing is the process of determining the **amount a customer must pay for a product or service.**

Objectives of Pricing

- Recover costs
- Generate profit
- Gain market share
- Compete effectively
- Reflect product value
- Encourage adoption

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Factors Affecting Pricing

1. Manufacturing cost

Higher production costs generally increase the minimum viable selling price.

2. Research and development cost

Medical device companies must recover development investment over time.

3. Competitor pricing

The company should understand prices of competing products.

4. Customer willingness to pay

Customers may pay more for significant clinical and economic benefits.

5. Clinical value

A product that significantly improves outcomes may justify a higher price.

6. Reimbursement

The available payment mechanism can strongly influence practical affordability and adoption.

7. Distribution cost

Distributor margins, transportation, installation, and service affect the final price.

Common Pricing Approaches

Cost-plus pricing:

Cost + desired profit margin

Competition-based pricing:

Price based on competitors

Value-based pricing:

Price based on perceived/ demonstrated customer and clinical value

Penetration pricing:

Lower initial price to encourage market adoption

Premium pricing:

Higher price for differentiated products with strong value or positioning

11. Product and Sales Support

Introduction

Product and sales support refers to the technical, clinical, marketing, training, and service assistance provided to customers and sales teams before and after a product is sold.

Product Support

Product support includes:

- Installation

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- User training
- Operation manuals
- Technical assistance
- Preventive maintenance
- Corrective maintenance
- Spare parts
- Software updates
- Troubleshooting
- Warranty service

Sales Support

Sales support helps sales personnel successfully communicate and demonstrate the product.

It includes:

- Product brochures
- Technical specifications
- Demonstration units
- Product presentations
- Training for sales personnel
- Competitor comparison
- Frequently asked questions
- Clinical evidence
- Customer case studies
- Pricing information

Importance

1. Improves customer confidence

Customers feel more confident when technical support is available.

2. Increases sales effectiveness

Sales personnel can answer technical and clinical questions.

3. Reduces product misuse

Proper training helps users operate devices correctly.

4. Improves customer satisfaction

Fast service and troubleshooting improve the customer experience.

5. Encourages repeat purchases

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Satisfied customers are more likely to purchase additional products.

Example

For an ultrasound system, sales support may include product demonstration and clinical application training, while product support includes installation, preventive maintenance, troubleshooting, and service.

12. Managing an Existing Product Line

Introduction

A **product line** is a group of related products marketed by the same company.

For example, a medical technology company may have a product line containing:

- Basic patient monitor
- Advanced patient monitor
- ICU monitor
- Wireless monitor

Managing an existing product line means continuously monitoring and improving the products to maintain their **market position, profitability, quality, and customer satisfaction**.

Major Activities

1. Monitor sales performance

Track:

- Sales volume
- Revenue
- Profit
- Market share
- Growth rate

2. Monitor customer feedback

Collect information about:

- Product satisfaction
- Complaints
- Usability
- Reliability
- Desired improvements

3. Product improvement

Improve:

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Hardware
- Software
- User interface
- Accuracy
- Connectivity
- Reliability

4. Manage product variants

Companies may offer different versions for different customer segments.

5. Manage pricing

Prices may need adjustment based on:

- Competition
- Cost
- Demand
- Market position

6. Inventory management

Maintain sufficient stock without creating excessive inventory.

7. Competitor monitoring

Track:

- New competitor products
- Technology changes
- Pricing
- Features
- Market strategy

8. Product life-cycle management

A product generally passes through:

Introduction → Growth → Maturity → Decline

At each stage, the company may modify its marketing and product strategy.

9. Decide whether to modify or discontinue

If a product has declining demand, high maintenance costs, or is replaced by better technology, the company may:

- Improve it
- Reposition it
- Reduce its price

MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Replace it
- Discontinue it

MEDICAL DEVICE MARKETING

Unit-01 Introduction

Introduction to Medical Device Marketing

Medical device marketing is the systematic process of identifying healthcare needs, developing suitable medical products, communicating their benefits, creating awareness among customers, and making the products available to hospitals, healthcare professionals, patients, and other users. Unlike ordinary consumer products, medical devices are associated with **patient safety, clinical effectiveness, regulatory requirements, reimbursement, and ethical responsibilities.**

The success of a medical device depends not only on its technical performance but also on whether it solves a genuine healthcare problem, satisfies the requirements of doctors and patients, receives necessary regulatory approvals, is economically viable, and can be purchased or reimbursed within the healthcare system.

Major activities in medical device marketing

1. **Identification of healthcare need** – Understanding an existing clinical or patient problem.
2. **Customer identification** – Identifying patients, doctors, hospitals, technicians, distributors, and other stakeholders.
3. **Market research** – Studying market size, competitors, customer requirements, and trends.
4. **Product development** – Designing a device that satisfies clinical and technical requirements.
5. **Regulatory approval** – Obtaining necessary permissions and certifications.
6. **Pricing** – Determining an appropriate and competitive price.
7. **Promotion** – Communicating the benefits and applications of the product.
8. **Distribution** – Making the product available through appropriate channels.
9. **Sales and follow-up** – Supporting customers after purchase.
10. **Reimbursement** – Understanding how healthcare providers and patients will pay for the product.

MEDICAL DEVICE MARKETING

Importance

Medical device marketing connects **technology with healthcare needs**. A technically excellent device may fail commercially if it is too expensive, difficult to use, unavailable through healthcare channels, or does not provide sufficient clinical value.

2. Customers

A **customer** is a person, organization, or institution that purchases, uses, influences the purchase of, or benefits from a product or service.

In healthcare, the customer is not always the same person as the end user. For example, a patient may use a medical device, while the hospital purchases it and a doctor recommends it.

Types of customers in healthcare

1. Patient

The patient is the ultimate beneficiary of most healthcare products and services. Their comfort, safety, affordability, and treatment outcomes are important.

2. Doctor/Physician

Doctors may recommend, prescribe, evaluate, or influence the selection of medical devices.

3. Hospital

Hospitals are major institutional customers. They may purchase equipment such as patient monitors, ultrasound systems, ventilators, and diagnostic devices.

4. Biomedical Engineer

Biomedical engineers evaluate the technical performance, maintenance requirements, compatibility, and safety of medical equipment.

5. Procurement Department

The procurement team handles purchasing, quotations, vendor comparison, negotiations, and purchase orders.

6. Nurses and Technicians

They are important users of many medical devices and can influence purchasing decisions based on usability and reliability.

MEDICAL DEVICE MARKETING

7. Distributors and Dealers

They purchase products from manufacturers and distribute them to hospitals, clinics, pharmacies, and other customers.

Customer needs

Customer needs may include:

- Safety
- Clinical effectiveness
- Reliability
- Ease of use
- Affordability
- Durability
- Technical support
- Training
- After-sales service
- Availability of spare parts
- Regulatory compliance

Example

For a **portable ECG device**:

- Patient → receives the benefit
- Doctor → interprets the ECG
- Nurse → may operate the device
- Hospital → purchases the device
- Biomedical engineer → evaluates and maintains it
- Distributor → supplies the device

MEDICAL DEVICE MARKETING

Thus, understanding the **different roles of customers** is essential for successful medical device marketing.

Decision Making

Introduction

Decision making is the process of identifying a problem or opportunity, collecting relevant information, evaluating alternatives, selecting the best option, implementing the decision, and evaluating its outcome.

In healthcare and medical device marketing, decision making is complex because decisions may involve **patient safety, clinical effectiveness, cost, regulatory compliance, technology, and ethical considerations**.

Steps in the Decision-Making Process

1. Identify the problem

The first step is to clearly understand what problem needs to be solved.

Example:

A hospital experiences delays in detecting abnormal oxygen levels in patients.

2. Define the objective

The organization should determine what it wants to achieve.

For example:

- Improve patient monitoring
- Reduce response time
- Improve accuracy
- Reduce operating costs

3. Collect information

Relevant information is collected from:

- Doctors
- Nurses

MEDICAL DEVICE MARKETING

- Patients
- Biomedical engineers
- Hospital management
- Research publications
- Competitors
- Market studies
- Regulatory authorities

4. Identify alternatives

Different possible solutions are generated.

For example:

- Purchase a conventional patient monitor
- Purchase a wireless patient monitoring system
- Develop an IoT-based monitoring system
- Upgrade the existing system

5. Establish decision criteria

Alternatives are evaluated using criteria such as:

- Cost
- Safety
- Accuracy
- Reliability
- Ease of use
- Clinical effectiveness
- Maintenance
- Regulatory compliance

MEDICAL DEVICE MARKETING

- Scalability
- Customer support
- **6. Evaluate alternatives**
- Each alternative is compared against the established criteria.
- For example:

Criteria	Device A	Device B	Device C
Cost	High	Medium	Low
Accuracy	High	High	Medium
Ease of use	Medium	High	High
Maintenance	High	Medium	Low
Connectivity	Medium	High	High

7. Select the best alternative

The alternative providing the best balance between **clinical value, technical performance, cost, safety, and customer requirements** is selected.

8. Implement the decision

The selected option is put into practice.

Implementation may include:

- Procurement
- Installation
- Training
- Testing
- Validation
- Documentation

9. Monitor the result

After implementation, performance should be monitored.

MEDICAL DEVICE MARKETING

Parameters may include:

- Device accuracy
- User satisfaction
- Patient outcomes
- Downtime
- Maintenance cost
- Return rate

10. Review and improve

If the decision does not produce the expected result, corrective action should be taken.

Factors Affecting Decision Making

Technical factors

- Accuracy
- Reliability
- Compatibility
- Performance
- Technology maturity

Economic factors

- Purchase cost
- Operating cost
- Maintenance cost
- Return on investment

Clinical factors

- Patient safety
- Clinical effectiveness

MEDICAL DEVICE MARKETING

- Ease of use
- Patient outcomes

Regulatory factors

- Regulatory approval
- Standards
- Quality requirements
- Documentation

Ethical factors

- Patient rights
- Privacy
- Informed consent
- Transparency

Organizational factors

- Budget
- Infrastructure
- Staff availability
- Management policy

Example

Suppose a hospital wants to purchase a new ultrasound machine. The decision-making process would involve:

Problem → Need identification → Market research → Identification of alternatives → Technical evaluation → Clinical evaluation → Cost comparison → Regulatory verification → Vendor evaluation → Purchase decision → Installation → Training → Performance evaluation.

MEDICAL DEVICE MARKETING

Buyer and Follow-Up

Buyer

A buyer is the person or organization responsible for purchasing a product or service. In healthcare, the buyer may be different from the actual user.

For example, a doctor may recommend a device, but the **hospital procurement department** may purchase it.

Types of healthcare buyers

1. Individual patients
2. Doctors and clinics
3. Hospitals
4. Government hospitals
5. Procurement departments
6. Distributors
7. Insurance organizations
8. Healthcare institutions

Buyer Decision Process

1. Identify the healthcare need
2. Search for available products
3. Compare alternatives
4. Evaluate technical and clinical performance
5. Evaluate price
6. Check regulatory approval
7. Negotiate with suppliers
8. Select the product
9. Purchase the product

MEDICAL DEVICE MARKETING

10. Evaluate the product after purchase

Follow-Up

Follow-up is the process of maintaining communication with a customer after initial contact, product demonstration, quotation, or purchase.

Importance of follow-up

- Builds customer relationships
- Identifies customer problems
- Provides technical support
- Improves customer satisfaction
- Encourages repeat purchases
- Helps identify new requirements
- Provides feedback for product improvement

Follow-up activities

Before purchase:

- Call the customer
- Provide product information
- Answer technical questions
- Provide quotation
- Arrange demonstrations

After purchase:

- Installation
- User training
- Performance verification
- Maintenance support
- Collect feedback

MEDICAL DEVICE MARKETING

- Handle complaints
- Provide warranty service

Example

After selling a patient monitor to a hospital, the company should contact the biomedical department to confirm installation, train users, check performance, and collect feedback.

Fundamentals of Reimbursement

Introduction

Healthcare reimbursement refers to the **payment made to healthcare providers, hospitals, clinics, or other healthcare organizations for providing healthcare services or products.**

It is an important component of healthcare economics because patients and healthcare providers may not directly bear the entire cost of treatment.

Major Participants

1. **Patient** – Receives healthcare.
2. **Healthcare provider** – Provides treatment or service.
3. **Hospital/Clinic** – Delivers healthcare services.
4. **Insurance company/payer** – May pay some or all eligible costs.
5. **Government** – Provides or finances healthcare through public schemes.
6. **Medical device company** – Supplies products used in treatment.

Basic Reimbursement Process

Patient receives treatment → Provider documents service → Claim is submitted → Payer evaluates claim → Claim is approved/rejected → Payment is made.

Types of Reimbursement

1. Fee-for-Service

Payment is made for individual healthcare services provided.

MEDICAL DEVICE MARKETING

2. Capitation

A fixed payment is provided for each patient for a defined period.

3. Bundled Payment

A single payment covers multiple services associated with a treatment or procedure.

4. Diagnosis-Related Payment

Payment is associated with the diagnosis or treatment category rather than each individual service.

5. Government-funded reimbursement

Government healthcare programs may reimburse hospitals or providers for eligible treatments.

Importance

Reimbursement affects:

- Product affordability
- Hospital purchasing decisions
- Patient access
- Medical device adoption
- Healthcare provider revenue
- Market potential

Example

A technologically advanced medical device may have excellent clinical performance but limited market adoption if patients or hospitals cannot afford it or if the relevant healthcare payment system does not cover its use.

MEDICAL DEVICE MARKETING

Healthcare Economics

Healthcare economics is the study of how **limited healthcare resources are allocated to satisfy the healthcare needs of individuals and society.**

Resources include:

- Money
- Medical equipment
- Healthcare professionals
- Medicines
- Hospital beds
- Infrastructure
- Time

Importance

Healthcare economics helps answer questions such as:

- Is the technology affordable?
- Does it provide sufficient clinical benefit?
- What is the cost of treatment?
- Is investment in the technology justified?
- How can limited healthcare resources be used efficiently?

Major concepts

Cost: Money or resources required for a healthcare product or service.

Benefit: Positive outcome obtained from healthcare intervention.

Cost-effectiveness: Comparison between cost and health outcome.

Cost-benefit: Comparison of monetary cost with monetary benefit.

Cost-utility: Comparison of cost with outcomes such as quality-adjusted life years.

Opportunity cost: The benefit lost when one option is selected instead of another.

MEDICAL DEVICE MARKETING

Example

A hospital may compare two patient-monitoring systems. One system may be cheaper initially, while another may have higher purchase cost but lower maintenance and better clinical performance. Healthcare economics helps determine which option provides greater overall value.

The Letter of Law, Ethics and Responsibilities

Introduction

Medical devices directly or indirectly affect human health and safety. Therefore, companies and healthcare professionals must comply with **laws, regulations, ethical principles, and professional responsibilities**.

The **letter of the law** refers to strict compliance with the actual legal requirements applicable to a product, organization, or activity. However, ethical responsibility goes beyond merely following minimum legal requirements.

A. Letter of Law

The letter of law means following established legal requirements exactly as specified by the relevant authorities.

For medical devices, legal requirements may cover:

- Product safety
- Quality management
- Manufacturing
- Clinical evaluation
- Regulatory approval
- Labelling
- Advertising
- Data protection
- Post-market surveillance

MEDICAL DEVICE MARKETING

- Reporting of adverse events

Why legal compliance is important

1. Protects patients
2. Ensures product safety
3. Builds customer confidence
4. Prevents legal penalties
5. Reduces product liability
6. Supports market acceptance
7. Protects the reputation of the organization

Example

A company should not market a medical device for a clinical indication that has not been legally authorized merely because the device appears technically capable of performing that function.

B. Ethics

Ethics refers to principles that guide people and organizations to distinguish between **right and wrong conduct**.

Important ethical principles include:

1. Beneficence

The product should provide benefit to the patient.

2. Non-maleficence

The product should not unnecessarily cause harm.

3. Autonomy

Patients should have the right to make informed decisions about their healthcare.

4. Justice

Healthcare resources and services should be provided fairly.

MEDICAL DEVICE MARKETING

5. Honesty

Manufacturers and healthcare professionals should provide accurate information.

6. Transparency

Limitations, risks, costs, and performance should not be deliberately hidden.

C. Responsibilities of Medical Device Companies

Product responsibility

The manufacturer should ensure:

- Safe design
- Reliable manufacturing
- Quality control
- Proper testing
- Appropriate documentation

Marketing responsibility

Companies should not:

- Make false claims
- Hide product limitations
- Mislead healthcare professionals
- Exaggerate clinical benefits
- Provide unsupported performance claims

Patient responsibility

Patient safety should remain a primary consideration.

Environmental responsibility

Companies should consider:

- Safe disposal

MEDICAL DEVICE MARKETING

- Electronic waste
- Sustainable materials
- Energy consumption
- Environmental impact

D. Responsibilities of Healthcare Professionals

Healthcare professionals should:

- Prioritize patient welfare
- Maintain confidentiality
- Use devices appropriately
- Provide accurate information
- Report adverse events
- Maintain professional competence
- Avoid conflicts of interest

E. Conflict of Interest

A conflict of interest occurs when personal, financial, or organizational interests may influence professional judgment.

Example:

A healthcare professional recommending a particular device because they receive an undisclosed financial benefit from the manufacturer represents an ethical concern.

F. Ethical Marketing

Ethical marketing requires:

1. Truthful product information
2. Evidence-based claims
3. Transparent pricing
4. No misleading advertisements

MEDICAL DEVICE MARKETING

5. Respect for patient privacy
6. Responsible promotion
7. Proper disclosure of risks
8. No inappropriate influence on purchasing decisions

Law vs Ethics

Law	Ethics
Legally mandatory	Based on moral/professional principles
Defined by legal authorities	Influenced by professional and societal values
Violation may result in legal penalties	Violation may damage trust and reputation
Represents minimum legal requirements	May demand a higher standard of conduct

Business Plan

Introduction

A business plan is a structured document that explains **what a business intends to achieve, how it will achieve it, who its customers are, what resources are required, and whether the business is financially viable.**

For a medical device company, a business plan connects the **healthcare problem → product → market → customer → financial model → implementation strategy.**

A typical business plan includes:

1. Executive summary
2. Problem identification
3. Product/service description
4. Customer identification
5. Market research
6. Competitor analysis

MEDICAL DEVICE MARKETING

7. Opportunity assessment
8. Marketing strategy
9. Financial analysis
10. Project scope
11. Risk analysis
12. Implementation plan
13. Stage-gate process

Market Research

Market research is the systematic process of collecting and analyzing information about **customers, competitors, market demand, pricing, trends, and business opportunities.**

Objectives

- Understand customer needs
- Estimate market size
- Identify competitors
- Determine customer preferences
- Understand pricing
- Identify market gaps
- Reduce business risk

Types of Market Research

Primary research

Information collected directly from customers.

Examples:

- Interviews
- Questionnaires
- Surveys

MEDICAL DEVICE MARKETING

- Focus groups
- Product demonstrations
- Hospital visits

Secondary research

Information obtained from existing sources.

Examples:

- Research papers
- Government reports
- Industry reports
- Published statistics
- Company websites
- Market databases

Market Research Process

Identify problem → Define research objective → Collect information → Analyze data →

Identify market opportunity → Make business decision

Example

For an **IoT-based sepsis monitoring device**, market research may determine:

- Number of hospitals requiring continuous monitoring
- Existing monitoring systems
- Doctors' requirements
- Device cost expectations
- Competitor products
- Required regulatory approvals
- Potential customers

MEDICAL DEVICE MARKETING

Accessing Opportunity / Assessing Opportunity

Assessing an opportunity means evaluating whether a particular idea or market need has sufficient **technical, clinical, commercial, and financial potential**.

Major factors

1. Problem

Is there a genuine healthcare problem?

2. Market size

How many potential customers exist?

3. Customer need

Are customers willing to use or purchase the solution?

4. Competition

Are similar products already available?

5. Competitive advantage

What makes the proposed product better?

6. Technical feasibility

Can the product actually be developed?

7. Regulatory feasibility

Can the product obtain necessary approvals?

8. Financial feasibility

Can the company develop and sell the product profitably?

9. Scalability

Can production be increased when demand grows?

10. Risk

What technical, financial, clinical, regulatory, and market risks exist?

Opportunity Assessment Example

MEDICAL DEVICE MARKETING

For a smart prosthetic socket:

Problem: Conventional sockets may cause discomfort.

Opportunity: Develop a sensor-based socket that monitors pressure and fit.

Customers: Amputees, rehabilitation centers, hospitals.

Technology: Pressure sensors + IoT + data analysis.

Market advantage: Real-time monitoring and personalized fitting.

Feasibility: Requires technical validation, clinical testing, manufacturing capability, and regulatory compliance.

Financial Analysis

Financial analysis evaluates whether a proposed business or project is **economically viable and financially sustainable**.

Major components

1. Initial investment

Money required to start the project.

Examples:

- Equipment
- Software
- Manufacturing setup
- Research and development
- Regulatory expenses
- Infrastructure

2. Operating costs

Costs required for regular operation.

Examples:

MEDICAL DEVICE MARKETING

- Salaries
- Electricity
- Maintenance
- Raw materials
- Marketing
- Transportation

3. Revenue

Income generated through product sales or services.

$$\text{Revenue} = \text{Selling Price} \times \text{Number of Units Sold}$$

4. Profit

$$\text{Profit} = \text{Revenue} - \text{Total Cost}$$

5. Break-even analysis

Break-even point is the point where total revenue equals total cost.

$$\text{Break-even units} = \text{Fixed Cost} \div (\text{Selling Price} - \text{Variable Cost per Unit})$$

6. Return on Investment

ROI measures the return obtained from an investment.

$$\text{ROI} = (\text{Net Return} \div \text{Investment}) \times 100$$

7. Cash flow

Cash flow represents the movement of money into and out of the business.

8. Financial risk

Possible risks include:

- Low sales
- Increased manufacturing cost
- Delayed regulatory approval

MEDICAL DEVICE MARKETING

- High R&D expenditure
- Competition
- Supply-chain problems

Example

If a medical device costs ₹10,000 to manufacture and is sold for ₹18,000, the difference contributes toward covering fixed costs and generating profit.

Project Scope

Project scope defines **what a project will and will not cover**.

It establishes the project's boundaries and prevents uncontrolled expansion of activities.

Components of project scope

1. Project objective
2. Deliverables
3. Requirements
4. Activities
5. Timeline
6. Resources
7. Budget
8. Responsibilities
9. Constraints
10. Exclusions
11. Acceptance criteria

Example

For an IoT patient monitoring project:

Objective: Develop a system for real-time monitoring of vital parameters.

MEDICAL DEVICE MARKETING

Inputs: Heart rate, SpO₂, temperature, etc.

Technology: Sensors + microcontroller + communication + cloud platform.

Output: Real-time display and alerts.

Deliverable: Functional prototype and validated monitoring system.

Exclusion: Full commercial regulatory approval may fall outside the initial prototype-development phase.

Importance

A clearly defined project scope:

- Controls cost
- Controls time
- Defines responsibilities
- Prevents scope creep
- Helps measure progress
- Improves project management

Stage Gate Process

Introduction

The **Stage-Gate process** is a structured product development methodology in which a project progresses through a series of **stages separated by decision gates**.

Each stage performs specific activities, while each gate evaluates whether the project should:

- Continue
- Stop
- Be modified
- Be delayed
- Be redirected

MEDICAL DEVICE MARKETING

It is particularly useful in medical device development because healthcare products involve significant **technical, clinical, regulatory, financial, and market risks**.

General Stage-Gate Model

Idea → Gate 1 → Concept → Gate 2 → Development → Gate 3 → Testing/Validation → Gate 4 → Launch → Gate 5 → Post-launch review

Stage 1 – Idea Generation

Potential product ideas are identified.

Sources include:

- Customer complaints
- Clinical problems
- Doctor feedback
- Research
- New technologies
- Market gaps

Gate 1: Determine whether the idea deserves further investigation.

Stage 2 – Concept Development

The selected idea is converted into a preliminary product concept.

Activities include:

- Customer identification
- Market research
- Competitor analysis
- Technical feasibility
- Initial risk assessment
- Preliminary business case

Gate 2: Decide whether the concept is sufficiently attractive and feasible.

MEDICAL DEVICE MARKETING

Stage 3 – Product Development

The actual product is designed and developed.

Activities include:

- Engineering design
- Prototype development
- Software development
- Component selection
- Risk management
- Design documentation

Gate 3: Evaluate whether the product is ready for testing and validation.

Stage 4 – Testing and Validation

The developed product is tested to determine whether it meets requirements.

Testing may include:

- Functional testing
- Safety testing
- Performance testing
- Usability testing
- Verification
- Validation
- Clinical evaluation where applicable

Gate 4: Decide whether the product is ready for commercialization.

Stage 5 – Product Launch

The product is introduced into the market.

Activities include:

MEDICAL DEVICE MARKETING

- Regulatory compliance
- Manufacturing
- Packaging
- Marketing
- Sales
- Distribution
- Customer training

Gate 5: Evaluate commercial performance.

Stage 6 – Post-Launch Review

After commercialization, the company evaluates:

- Sales
- Customer satisfaction
- Product complaints
- Clinical performance
- Maintenance
- Profitability
- Adverse events
- Opportunities for improvement

Decision Criteria at Gates

- Each gate may consider:

Factor	Question
Market	Is there sufficient demand?
Customer	Does it solve a real customer problem?
Technical	Can it be developed successfully?
Clinical	Does it provide meaningful healthcare benefit?

MEDICAL DEVICE MARKETING

Regulatory	Can it satisfy applicable requirements?
Financial	Is it economically viable?
Risk	Are risks acceptable?
Competition	Does it have sufficient competitive advantage?

Advantages of Stage-Gate Process

1. Reduces risk

Problems can be identified before large investments are made.

2. Controls expenditure

Projects that are unlikely to succeed can be stopped early.

3. Improves product quality

Each stage has defined requirements and evaluation criteria.

4. Supports informed decisions

Management receives structured information before approving the next stage.

5. Improves resource allocation

Money, people, equipment, and time can be directed toward promising projects.

6. Supports regulatory planning

Medical device regulatory and quality requirements can be considered throughout development.

7. Improves commercialization

Market and customer requirements are considered before product launch.

Example – Smart Patient Monitoring Device

Idea: Develop a wearable device for continuous monitoring.



Concept: Identify parameters such as heart rate, SpO₂, temperature, and activity.



Development: Develop sensors, embedded system, communication, and software.



Testing: Verify accuracy, safety, usability, and connectivity.



Regulatory/Commercial Gate: Evaluate compliance and business viability.



Launch: Manufacture and sell the product.



MEDICAL DEVICE MARKETING

Post-launch: Collect customer feedback and monitor product performance.



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)

Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A" Grade



JAVA PROGRAMMING

BY

DR. R. RAMALAKSHMI

DR. K. MURUGESWARI

MR. M. K. NAGARAJAN

MRS. A. M. GURUSIGAAMANI

Course Outline



CO 1. Understand the object oriented concepts using java



CO2. Apply fundamental programming concepts of Java to develop stand alone applications



CO 3. Solve Real World Problem through reusable and error free code.



CO 4. Design and develop distributed applications.



CO 5. Implement window based applications using event handling mechanisms

Course description

Java still has its importance and popularity in the software industry, in spite of recent development of several high level languages. It has an excellent support of high-level as well as low-level functionality, which makes it suitable for many applications. This course provides students with a comprehensive study of the Java programming language. Instead of Classroom lectures, practical classes stress the strengths of Java, which provide the students with the means of writing efficient, maintainable, and portable code. A course with mini project is one of the great learning opportunities to develop students in various aspects. This course will also help the students during their placement sittings, as most of the companies test proficiency in programming using Java.

UNIT I OOP IN JAVA & INHERITANCE

Object Oriented Programming Concepts - OOP in Java – Characteristics of Java – Fundamental Programming Structures in Java – Defining classes in Java – Comments, Data Types, Variables, Operators, Control Flow, Arrays - constructors, methods -access specifiers - static members – Packages- Inheritance – Super classes- sub classes –Protected members – constructors in sub classes- Strings.

UNIT II EXCEPTION HANDLING AND I/O

Exceptions - exception hierarchy - throwing and catching exceptions – built-in exceptions, creating own exceptions, Stack Trace Elements-Input / Output Basics – Streams – Byte streams and Character streams – Reading and Writing Console – Reading and Writing Files - abstract classes and methods - final methods and classes

UNIT III INTERFACES AND MULTITHREADING

Interfaces – defining an interface, implementing interface, differences between classes and interfaces - extending interfaces - Object cloning -inner classes- Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads, Inter-thread communication, daemon threads - Generic Programming.

UNIT IV AWT AND EVENT DRIVEN PROGRAMMING

AWT Event Hierarchy- Components - Graphics programming – Applets-Frame – working with 2D shapes - Using color, fonts, and images - Basics of event handling - event handlers - adapter classes - actions - mouse events - Introduction to Swing – layout management - Swing Components – Windows – Menus – Dialog Boxes.

UNIT V NETWORKING AND JDBC

Networking Basics - The Networking Classes and Interfaces -TCP/IP Client Sockets URL TCP/IP Server Sockets – **Datagrams** - A Relational Database Overview - JDBC Introduction - JDBC Product Components - JDBC Architecture - Case studies.



Unit 1

Outcomes

Define classes.

Create objects using default and parameterized constructors.

Run stand alone applications using programming constructs of Java

Inherit the properties of an existing classes.

Hide the members of a class using access specifiers

Use predefined packages and create own package to group related classes.

Unit 1 Outline



Lesson 1. Object Oriented programming Concepts



Lesson 2. Fundamental programming Constructs in Java



Lesson 3. Access Specifiers



Lesson 4. Constructors



Lesson 5. Inheritance



Lesson 6. Packages

Briefs about OOP concepts, Characteristics of Java , Fundamental Programming Structures in Java , Defining classes in Java, Comments, Data Types, Variables, Operators, Control Flow, Arrays, constructors, methods, access specifiers, static members, Packages, Inheritance, Super classes, sub classes, Protected members, constructors in sub classes, Strings.

Course Progress (use before starting new lesson)

Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance



Introduction

- Object-Oriented Programming is a programming paradigm that uses “Objects” and their interactions to design applications.
- It simplifies the software development and maintenance by providing some concepts.



Object Oriented Programming Concepts

- . Object
- . Class
- . Abstraction
- . Encapsulation
- . Inheritance
- . Polymorphism



Object

- . An object is defined as the Realworld Entity.
- . It consists of properties, tasks performed by objects.
- . It also defined as instance of a Class

Ex: Human

Properties ☐ Name, Colour, Height

Tasks Performed ☐ Walk(), Run(), Work(), Read()

Creating an object

Creating an object of the class is also called as instantiating an object. Here, we create an object called “s” and print the value of regno.

```
public class student1
{
    int regno=5;
    public static void main(String args[])
    {
        student1 s=new student1();
        System.out.println(s.regno);
    }
}
```


OUTPUT OF OBJECT CREATION

- Output

```
C:\Users\admin\Desktop\dhilipjava>javac student1.java
```

```
C:\Users\admin\Desktop\dhilipjava>java student1  
5
```

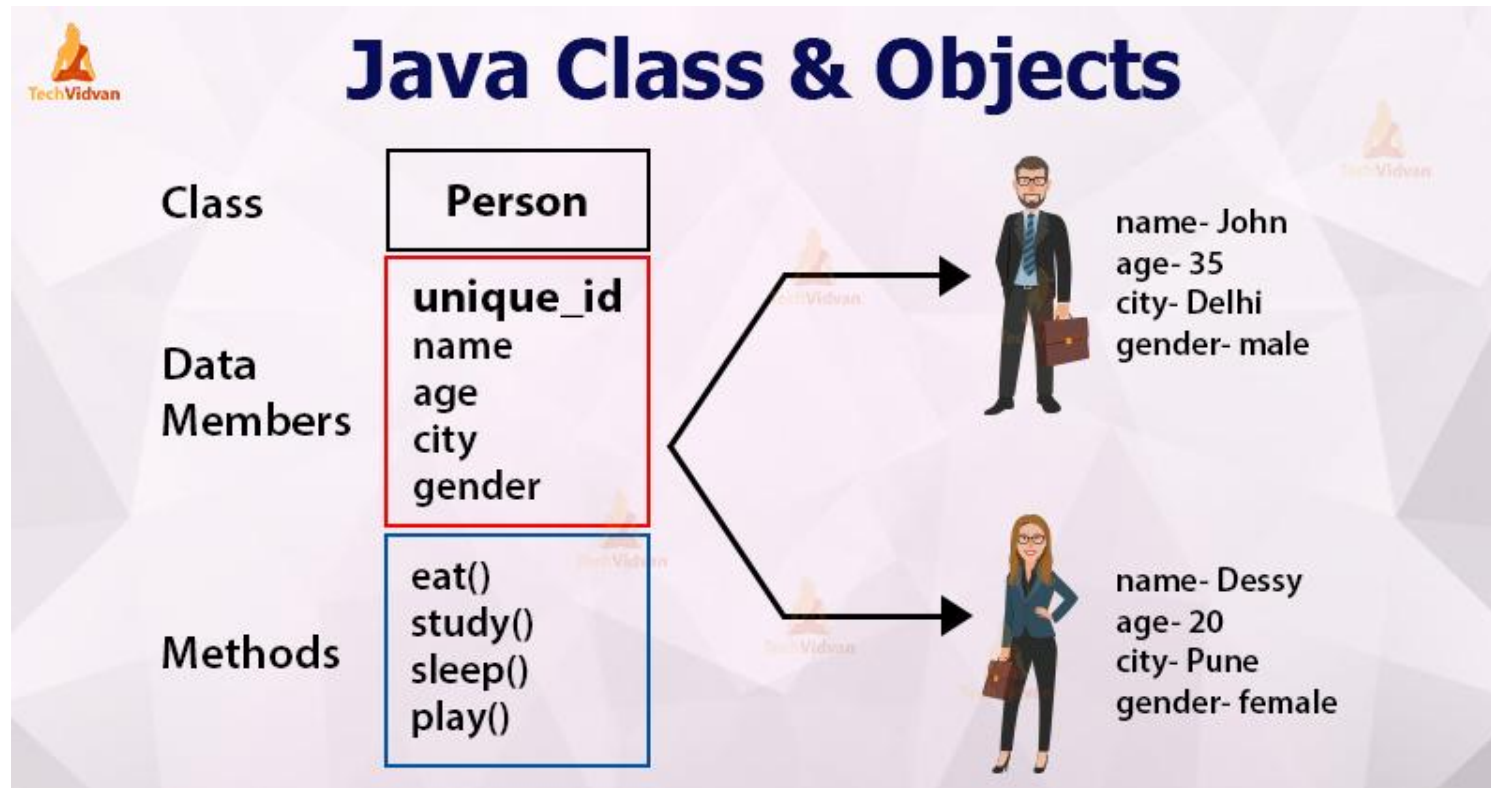
```
C:\Users\admin\Desktop\dhilipjava>_
```



Class

- ❖ Class is a collection of data members and member functions.
- ❖ It is called as the blueprint that defines the variables and methods.
- ❖ When creating instance of a class the system allocates enough memory for the object.

Java Class & Objects



source:techvidvan.com

Syntax of Class:

```
class class_name
{
    type instance_variable;
    type methodname1(parameter_list)
    {
        // body of method
    }
}
```

Example of a Class

```
Class Student
{
    int regno,marks;
    char name;
    void totalmarks ()
    {
        .....
    }
}
```

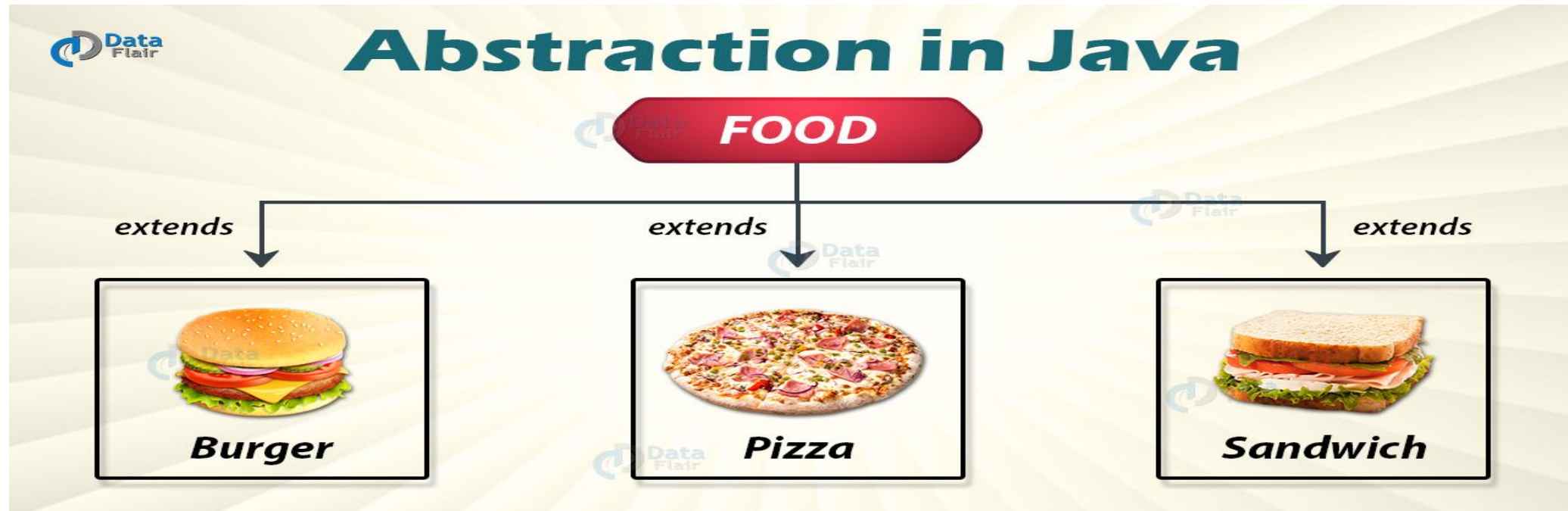


Abstraction

It shows only essential things and hides unnecessary information from the user. It helps the user in reducing programming complexity. Two way of abstraction are as follows

- ❖ Data Abstraction
- ❖ Process Abstraction

Abstraction



Source: data-flair.training.com

Example of Abstraction

```
abstract class Bike
{
    abstract void run();
}
class Honda4 extends Bike
{
    void run()
    {
        System.out.println("running safely");
    }
    public static void main(String args[])
    {
        Bike obj = new Honda4();
        obj.run();
    }
}
```


Output of Abstraction

```
C:\Users\admin\Desktop\dhilipjava>javac Honda4.java
```

```
C:\Users\admin\Desktop\dhilipjava>java Honda4  
running safely
```

```
C:\Users\admin\Desktop\dhilipjava>
```



Encapsulation

- ❖ It is a mechanism of wrapping data(variable) and code acting on the data(methods) together as a single unit.
- ❖ It binds together the code and the data it manipulates and keeps both safe from outside.
- ❖ In Java, the **basis of encapsulation is the class.**
- ❖ A **class** defines the structure and behavior (data and code) that will be shared by a set of objects.

Encapsulation Figure

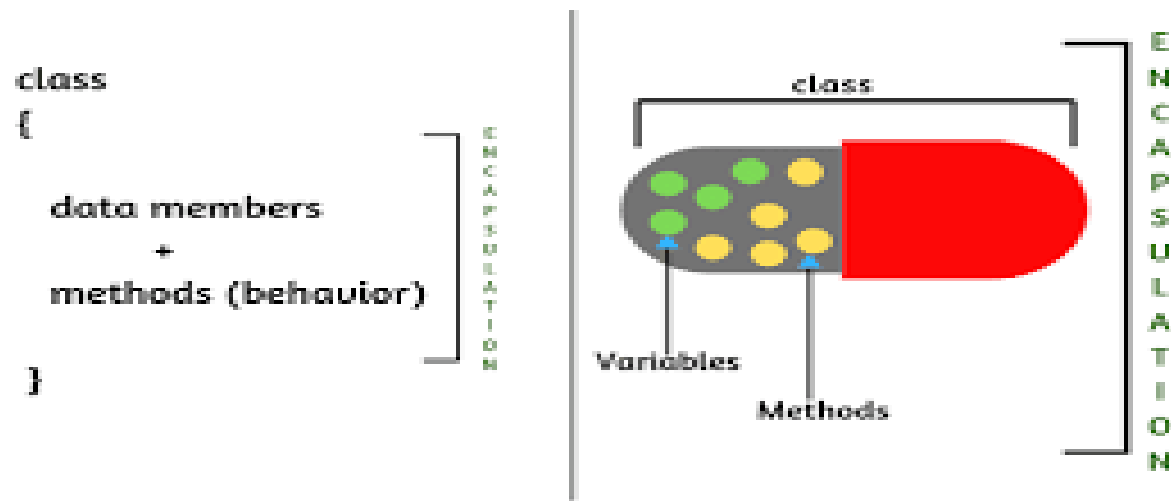


Fig: Encapsulation

Image src:medium.com



Imagesrc:codeeasy.net

Program for Encapsulation

```
public class RunEncap
{
    public static void main(String args[])
    {
        EncapTest encap = new EncapTest();
        encap.setName("James");
        encap.setAge(20);
        encap.setIdNum("12343ms");

        System.out.print("Name : " + encap.getName() + " Age : " + encap.getAge());
    }
}
```

Output of Encapsulation

```
C:\Users\admin\Desktop\dhilipjava>javac EncapTest.java  
C:\Users\admin\Desktop\dhilipjava>javac RunEncap.java  
C:\Users\admin\Desktop\dhilipjava>java RunEncap  
Name : James Age : 20  
C:\Users\admin\Desktop\dhilipjava>
```

Inheritance

- It is the process of deriving a class from the base class.
- For example, one person is getting the behavior of his parent is an example of inheritance.
- If one object acquires the properties of another object.
- This is important because it supports the concept of hierarchical classification.
- Types of inheritance are as follows
 - Single Inheritance
 - Multiple Inheritance
 - Multilevel Inheritance
 - Hierarchal Inheritance
 - Hybrid Inheritance

Program for Inheritance

```
class Doctor
{
    void Doctor_Details()
    {
        System.out.println("Doctor Details...");
    }
}
class Surgeon extends Doctor
{
    void Surgeon_Details()
    {
        System.out.println("Surgen Detail...");
    }
}
```

```
public class Hospital
{
    public static void main(String args[])
    {
        Surgeon s = new Surgeon();
        s.Doctor_Details();
        s.Surgeon_Details();
    }
}
```

Output of Inheritance

```
NAME : DINESH NGC : 20
C:\Users\admin\Desktop\dhilipjava>javac Hospital.java

C:\Users\admin\Desktop\dhilipjava>java Hospital
Doctor Details...
Surgen Detail...

C:\Users\admin\Desktop\dhilipjava>_
```




Polymorphism

- Polymorphism “Poly” means many “morph” means form.
- It is one of the feature in oops that performs single action in different ways. For example class vehicle has method cars().
- In that cars are of different types XUV, ZEDON, HATCHBACK etc
- Two types of polymorphism are there. They are as follows
 - Compile time polymorphism (Static Binding)
 - Run time polymorphism (Dynamic Binding)
- Another one good example is that a person at the same time can have different characteristics. A man act as a father, son, husband and an employee.

Polymorphism



Imagesrc:csharp-station.com

Program for Polymorphism

```
class MultiplyFun
{
    static int Multiply(int a, int b)
    {
        return a * b;
    }
    static double Multiply(double a, double b)
    {
        return a * b;
    }
}
class Main5
{
    public static void main(String[] args)
    {
        System.out.println(MultiplyFun.Multiply(2, 4));
        System.out.println(MultiplyFun.Multiply(5.5, 6.3));
    }
}
```

Output of Polymorphism

```
C:\Users\admin\Desktop\dhilipjava>javac Main5.java  
  
C:\Users\admin\Desktop\dhilipjava>java Main5  
8  
34.65
```



What is JAVA

- ❖ Java is a high level, class-based object-oriented programming language.
- ❖ It is a high-level programming language which right once and run anywhere.
- ❖ When a programmer writes a java application, then compiled code and he can run that on most operating system (OS) like windows, Linux and MAC OS.
- ❖ It is used to build large enterprise class application.
- ❖ It is used for building large scale system. It is a software only platform running on top of hardware-based platform.
 - ❖ Java Virtual Machine (JVM)
 - ❖ Java Application Programming Interface (Java API)



Characteristics of JAVA

SIMPLE

It is easy for specialized programmer to learn and use.

SECURE

It is providing security while creating internet application

PORTABLE

It can be executed in any environment where java should be there

OBJECT ORIENTED

It is clean, usable, pragmatic approach to objects, not restricted by the need for compatibility with other languages.



Characteristics of JAVA

ROBUST

Java encourage error free programming by doing runtime checking

MULTITHREADED

Java support multithreaded programming

ARCHITECTURAL NEUTRAL

It is not knotted to a specific machine or operating system architecture.

INTERPRETED

Using java bytecode, it supports cross platform code



Characteristics of JAVA

- **HIGH PERFORMANCE**

The Java enables high performance using just in time compiler

- **DISTRIBUTED**

Java was designed for distributed environment of the Internet

- **DYNAMIC**

Java is more dynamic than c and c++, as it is designed to adapt the evolving environment.

Java programs carry substantial amounts of run-time type information that is used to verify and resolve accesses to objects at run time.



OOP in JAVA

Java defines eight simple data types:

- 1) byte-8-bit integer type
- 2) short-16-bit integer type
- 3) int-32-bit integer type
- 4) long-64-bit integer type
- 5) float-32-bit floating-point type
- 6) double-64-bit floating-point type
- 7) char-symbols in a character set
- 8) boolean-logical values

Data Types

DATA TYPES	RANGE	EXAMPLE	USAGE
byte-8-bit integer type	-128 TO 127	Byte b=-15	When working with data streams
short-16-bit integer type	-32768 to 32767	short c= 1000	Least used simple type
int-32-bit integer type	-2147486648 to 2147483647	int b=-600000	It is common one, used to control loops and to index array
long:64-bit integer type	-9223372036854775808 to 9223372036854775807.	long d = 506000000000000078;	When int type is not large enough to hold values

Data Types

float-32-bit floating-point type	$1.4e^{-045}$ to $3.4e^{+038}$	Float d=3.5	Fractional part is used and large degree of accuracy is not needed
double-64-bit floating-point type	$4.9e^{-324}$ to $1.8e^{+308}$	double pi=3.14	Handling large valued number
char-symbols in a character set	0 to 65536	Char a='a'	It is used to represent both ASCII and Unicode character set
Boolean-logical values	True or false	boolean b=(2<3)	It returns relational operators such as 1<2



Variables

It uses variable to store data.

TO allocate memory JVM requires to specify the datatype of the variable.

To associate an identifier of the variable.

It is assigned with initial value.

All are processed as a part of variable declaration.

Variables

declaration	It is used to assign the type of variables
initialization	How to give starting value of variable
scope	How the variables are visible to other parts of the program
lifetime	How the variable is created used and destroyed
type conversion	How automatic type conversion is handled in java
type casting	How variable type is narrowed down



Variable Declaration With Example

Syntax

`datatype identifier=value;`

Example

`int a=20;`

`float i=20.5;`

`double pi=3.14;`

Arrays

- ❖ Array is a collection of related data of similar type.
- ❖ The values can be stored in large amount based on the desire of the user.
- ❖ Arrays can be declared, created, initialized and used.
- ❖ Array is of two types as are shown below
 - One dimensional array
 - Multi dimensional array

Array Initialization

Arrays can be initialized when they are declared:

```
int monthDays[] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

Note:

There is no need to use the new operator

There array is created large enough to hold all specified elements

Array Syntax

Two types of declaring array are as follows

```
type array_variable[];
```

```
type [] array_varriable;
```

After creating an array it does not actually exist. In order to create an array “new” operator should be used.

- `array_variable = new type[size]`

This create new variable to hold size elements of type, which reference will be kept in array variable. Here array variable index starts with zero. Java run time system makes sure that all array indexes are in correct range otherwise it leads to runtime error



Single Dimensional array

Example

```
int days[]={1,2,3,4,5,6,};
```

This is direct way of giving values to array variable. And in this no need to specify the size of the array as we will do in c or c++.

In array concept, if more than one set bracket is there we will call it as multidimensional array. It is also called as array of array.



Multidimensional Array

`int array[][]; //Declaration`

`int array=new int[4][5]; //Creation`

This represents 4 rows and 5 columns.

`int array[][]={{2,3,4},{5,6,7}}// initialization`

This array represents 2 rows and 3 columns



Operator Types

Java operators are used to build value expression.

The listed below are some of the operators used in java

- Arithmetic Operator
- Logical Operator
- Assignment Operator
- Relational Operator
- Bitwise Operator

Arithmetic Operator

Operator	Purpose	a=10,b=3	a=12.5,b=2.0	a='P' b='T'
+	Addition	13 (a+b)	14.5	164
-	Subtraction	7 (a-b)	10.5	-4
*	Multiplication	30 (a*b)	25.0	6720
/	Division	3 (a/b)	6.25	0
%	Modulo Division	1 (a%b)	----	80

Example

a=5 and b=-2 then a%b=1

a=-5 and b=2 then a%b=-1



Logical Operators

LOGICAL OPERATORS :(also called logical connectives)

1. && (logical and)
2. || (logical or)

These 2 operators are used to combine the individual logical expressions into more complex conditions

Results of logical and (&&)

- The result of the logical and (&&) operation will be true only if both operands are true.

Expression 1	Expression2	Result
1	1	1
1	0	0
0	1	0
0	0	0

Results of logical or (||)

- The result of the logical or (||) operation will be true if either operand is true.

Expression 1	Expression 2	Result
1	1	1
1	0	1
0	1	1
0	0	0



Logical Not (!)

- The logical not (!) operator requires only one operand which represents a logical value 0 or non zero.
- The **result** of the operation is **non zero** if and only if the **operand is zero**.
- **Example**
 - $i=7$, and $!(i>6)$ then result is 0.

ASSIGNMENT OPERATORS:

- ✓ Assignment operators are used to assign a value in a particular variable.
- ✓ The most commonly used assignment operator is '='

Assignment operator form

Identifier= expression

- ✓ Identifier may be variable
- ✓ Expression may be constant, variable, or a more complex expression
- ✓ Assignment expressions are referred to as an assignment statement.
- ✓ Ex: a=5;

Multiple assignment

- Multiple assignment of the form:
- **Identifier1 =Identifier2=expression**
- Is equivalent to
- **Identifier1 = (Identifier2=expression)**
- **Example**
- **a=b=5;** It means first 5 is assigned to 'b' and then the value of b (that is 5) is assigned to 'a'.



SHORTHAND ASSIGNMENT

SHORTHAND ASSIGNMENT OPERATORS:

Syntax:

Var operator=exp;

1) += 2) -= 3) *=, 4) /=, 5) %= these are the shorthand operators.

Example

var +=exp is equivalent to var=var+exp

var-=exp is equivalent to var=var-exp

For Ex:

If i=5,j=7

j*=(i-3)

j=j*(i-3) then j=7*(5-3) then j=7*2 finally j=14.

i%=j

i=i%j then i=5%7 then i=5



RELATIONAL OPERATOR:

There are 4 relational operators

- ❖ < (Less than)
- ❖ > (Greater than)
- ❖ <= (Less than or equal to)
- ❖ >= (Greater than or equal to)

There are 2 Equality operators

- ❖ == (equal to)
- ❖ != (not equal to)

➤ These 6 operators are used to form logical expressions, which represent conditions that are either true or false.

RELATIONAL OPERATOR:

- The resulting expressions will be of type integer, since true is represented by integer value 1 and false is represented by the value 0.

Example

$i=1, j=2, k=3$

Expression	Result
$i < j$	1
$(i+j) \geq k$	1
$k \neq 3$	0



BITWISE OPERATORS:

Bitwise operator used for manipulation of data at bit level.

OPERATOR	MEANING
&	Bitwise AND
	Bitwise OR
^	Bitwise Exclusive OR
<<	Shift left
>>	Shift Right
~	One's complement

Expression

The data type of the value returned by an expression depends on the elements used in the expression. The expression

- `number = 0;`

which returns an int because the assignment operator returns a value of the same data type as its left-hand operand; in this case, number is an int.

In Java, compound expressions also be used. The data type required by one part of the expression matches the data type of the other. The example of a compound expression is as follows

- `1 * 2 * 3`

Control statements

- ❖ A program is a sequence of instructions.
- ❖ Java control statements cause the flow of execution to advance and branch based on the changes to the state of the program.
- ❖ Each instruction is executed only once in the order it appears.
- ❖ Control statements are divided into three groups:
 1. Selection statements
 2. Iteration statements
 3. Jump statements



Selection Statements

One of the several possible branches will then be carried out, depending on the outcome of the logical test. This is known as selection statement

- if
- if-else
- nested if-else
- switch

if Statement

- In if statement, only one condition is checked.
- If the condition is true the statement is executed. If the condition is false it will exit out of the loop

```
class IfStatement
{
    public static void main(String[] args)
    {
        int number = 10;
        // checks if number is greater than 0
        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
        System.out.println("Statement outside if block");
    }
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>javac IfStatement.java
```

```
C:\Users\admin\Desktop\dhilipjava>java IfStatement
```

```
The number is positive.
```

```
Statement outside if block
```

If else Statement

- In this if else statement, if the condition is true then, the true statement was executed or else the else part is executed.
- Or else the statement outside the block was executed.

```
class ifelsestmt
{
    public static void main(String[] args)
    {
        int number = 10;
        // checks if number is greater than 0
        if (number > 0)
        {
            System.out.println("The number is positive.");
        }
        // execute this block
        // if number is not greater than 0
        else
        {
            System.out.println("The number is not positive.");
        }
        System.out.println("Statement outside if...else block");
    }
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>javac ifelsestmt.java  
  
C:\Users\admin\Desktop\dhilipjava>java ifelsestmt  
The number is positive.  
Statement outside if...else block
```

Switch Statement

- The switch statement allows us to execute a block of code among many alternatives.

```
class switchdemo
{
    public static void main(String[] args)
    {
        int number = 44;
        String size;
        // switch statement to check size
        switch (number)
        {
            case 29:
                size = "Small";
                break;
            case 42:
                size = "Medium";
                break;
            // match the value of week
            case 44:
                size = "Large";
                break;
            // ...
        }
    }
}
```

```
        case 48:
            size = "Extra Large";
            break;
    }
    System.out.println("Size: " + size);
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>javac switchdemo.java
```

```
C:\Users\admin\Desktop\dhilipjava>java switchdemo
```

```
Size: Large
```




ITERATION STATEMENT

During looping, a set of statements are executed until some conditions for termination of the loop is encountered.

A program loop therefore consists of two segments one known as body of the loop and other is the control statement.

The control statement tests certain conditions and then directs the repeated execution of the statements contained in the body of the loop.

Java provides three iteration statements

- while
- do -while
- for



Jump statements

It enables transfer of control to other parts of the program. In java we are having three types of jump statements

- break
- continue
- return



TYPE CONVERSION

- Conversion in java is done in two ways
 1. Implicit type conversion- Carried out by compiler automatically
 2. Explicit type conversion- Carried out by program using casting
- Size Direction of Data Type
 - - Widening Type Conversion (Casting down)
Smaller Data Type -> Larger Data Type
 - - Narrowing Type Conversion (Casting up)
Larger Data Type -> Smaller Data Type



TYPE CASTING

It is the of assigning a primitive data type value to another data type value. It is also a casting of a class or interface into another class or interface.

Syntax

(target type) values

Example

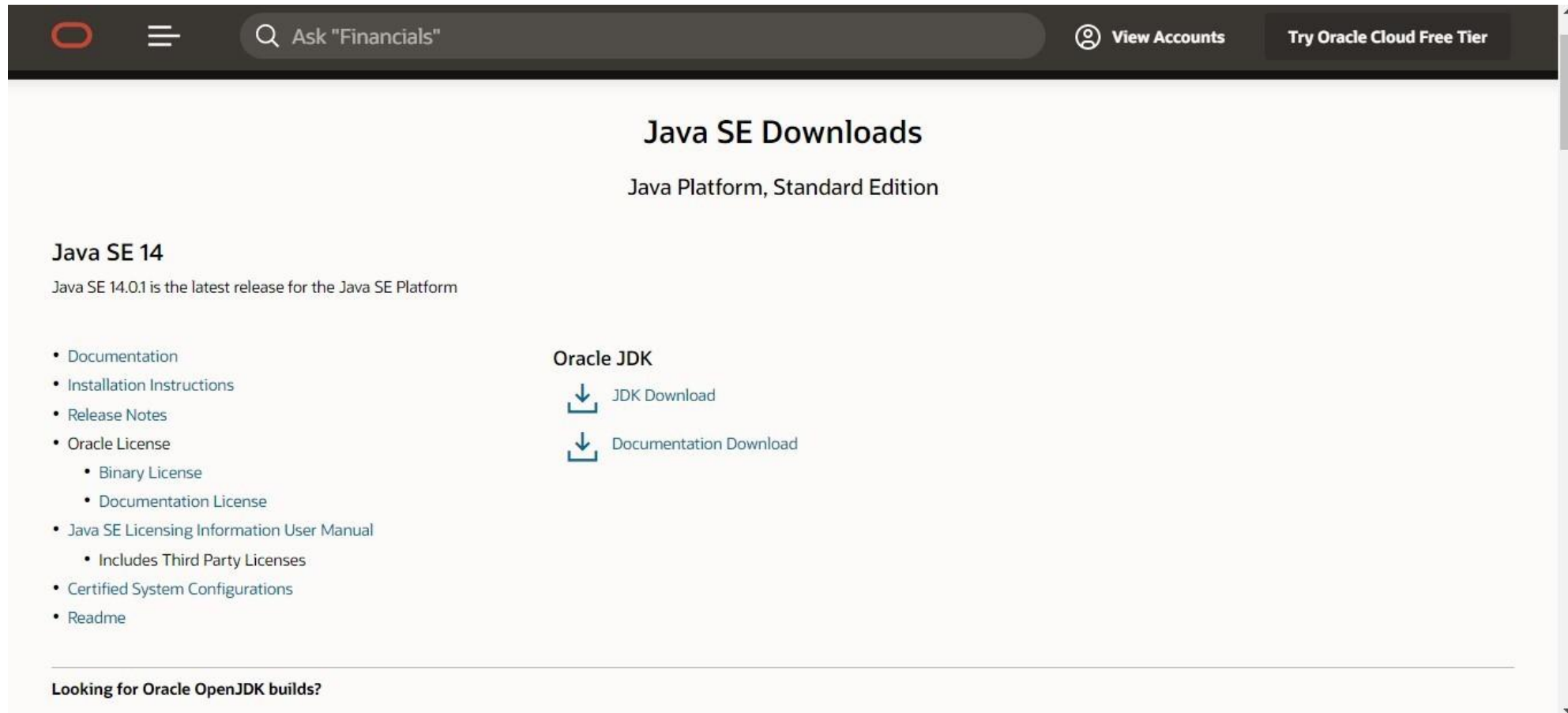
- integer value will be reduced module bytes range:
- int a;
- byte a = (byte) i;
- floating-point value will be truncated to integer value:
 - float a;
 - int b = (int) a;



How to download and install Java for 64-bit machine

The following steps can be followed in order to download and install java. All the steps are described below has been performed on the Windows 10 operating system, but the procedure is quite similar to other operating systems as well.

Step 1: Open <https://www.oracle.com/java/technologies/javase-downloads.html> url in the browser and it will navigate to the official Oracle Java downloads page.

The screenshot shows the Oracle Java SE Downloads page. The header includes the Oracle logo, a search bar with the text 'Ask "Financials"', and links for 'View Accounts' and 'Try Oracle Cloud Free Tier'. The main heading is 'Java SE Downloads' with the subtitle 'Java Platform, Standard Edition'. Under 'Java SE 14', it states 'Java SE 14.0.1 is the latest release for the Java SE Platform'. A list of links on the left includes 'Documentation', 'Installation Instructions', 'Release Notes', 'Oracle License' (with sub-links for 'Binary License' and 'Documentation License'), 'Java SE Licensing Information User Manual' (with a sub-link for 'Includes Third Party Licenses'), 'Certified System Configurations', and 'Readme'. On the right, under 'Oracle JDK', there are two download links: 'JDK Download' and 'Documentation Download', each with a download icon. At the bottom, there is a link for 'Looking for Oracle OpenJDK builds?'.

Step 2: Now, scroll to the version of the Java which we want to download and click on **JDK Download** option as shown below:



Java SE Downloads

Java Platform, Standard Edition

Java SE 14
Java SE 14.0.1 is the latest release for the Java SE Platform

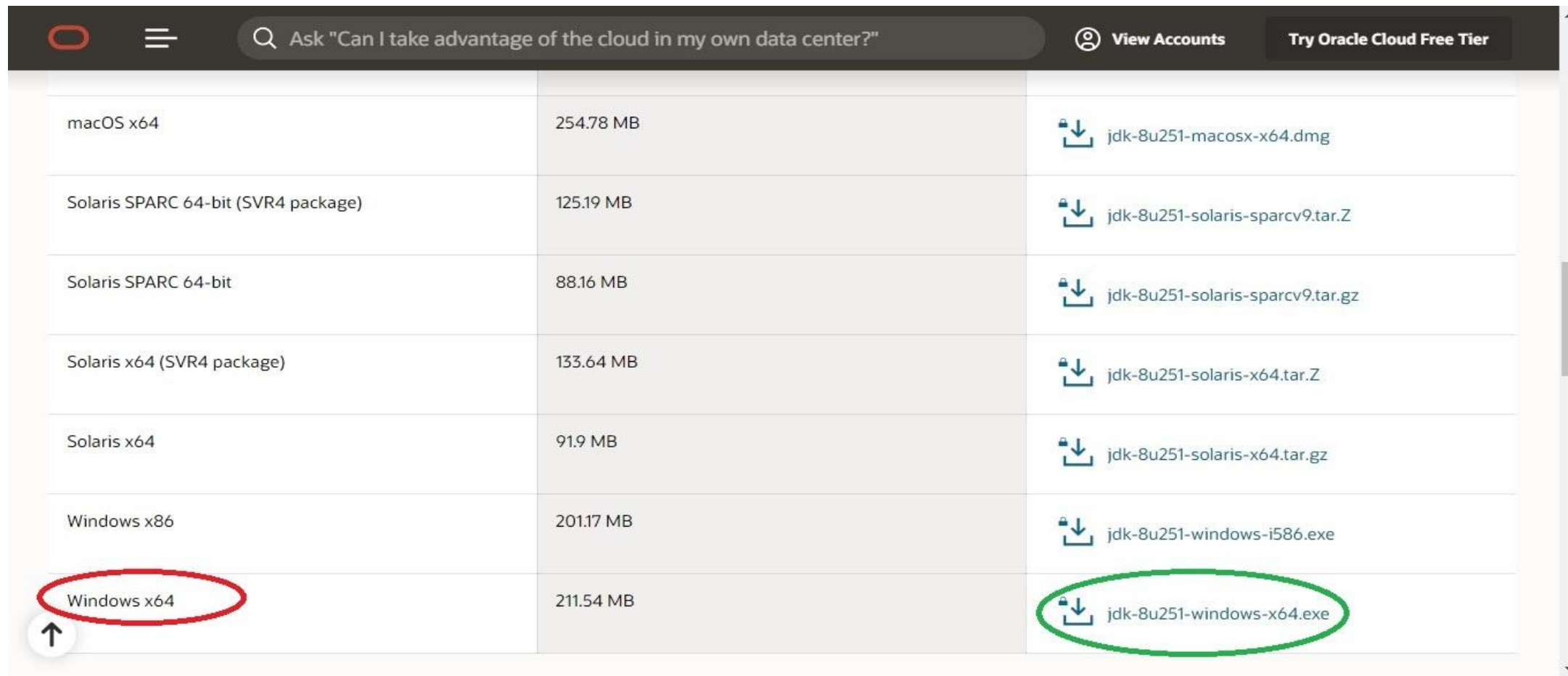
- Documentation
- Installation Instructions
- Release Notes
- Oracle License
 - Binary License
 - Documentation License
- Java SE Licensing Information User Manual
 - Includes Third Party Licenses
- Certified System Configurations
- Readme

Oracle JDK

- ↓ JDK Download
- ↓ Documentation Download

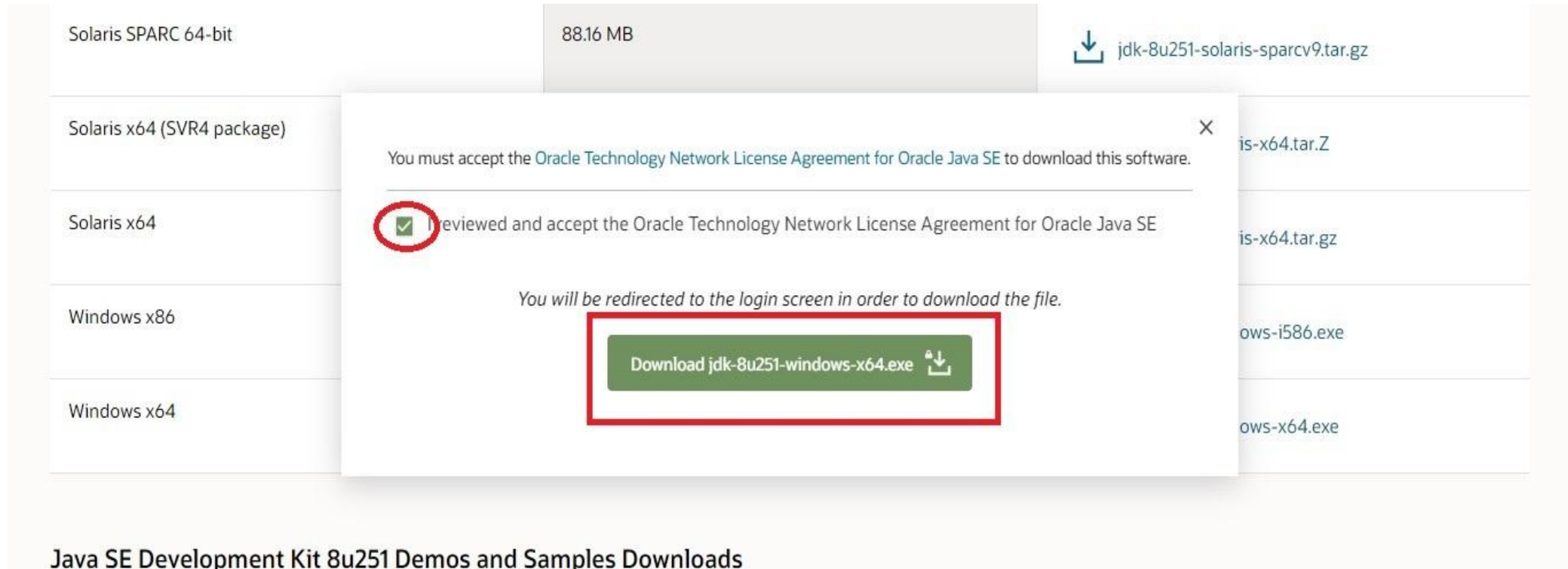
Looking for Oracle OpenJDK builds?

Step 3: Scroll down to the page and click on the download button option suitable for your computer Operating system. But for a 64-bit machine, choose the software name ending with **x64**.

The screenshot shows the Oracle Java download page. At the top, there is a search bar with the text 'Ask "Can I take advantage of the cloud in my own data center?"', a 'View Accounts' button, and a 'Try Oracle Cloud Free Tier' button. Below this is a table of download links for different operating systems. The 'Windows x64' row is highlighted with a red circle, and the corresponding download link 'jdk-8u251-windows-x64.exe' is circled in green. An upward arrow icon is visible next to the 'Windows x64' row.

macOS x64	254.78 MB	 jdk-8u251-macosx-x64.dmg
Solaris SPARC 64-bit (SVR4 package)	125.19 MB	 jdk-8u251-solaris-sparcv9.tar.Z
Solaris SPARC 64-bit	88.16 MB	 jdk-8u251-solaris-sparcv9.tar.gz
Solaris x64 (SVR4 package)	133.64 MB	 jdk-8u251-solaris-x64.tar.Z
Solaris x64	91.9 MB	 jdk-8u251-solaris-x64.tar.gz
Windows x86	201.17 MB	 jdk-8u251-windows-i586.exe
Windows x64	211.54 MB	 jdk-8u251-windows-x64.exe

After clicking on the download button, a popup will appear which says that we have to accept **Oracle Technology Network License Agreement for Oracle Java SE** in order to download this software. Therefore, click on the checkbox and then proceed to download as shown below:

The screenshot shows the Oracle Java SE download page. On the left, there is a list of operating systems: Solaris SPARC 64-bit (88.16 MB), Solaris x64 (SVR4 package), Solaris x64, Windows x86, and Windows x64. On the right, there are download links for various file formats: jdk-8u251-solaris-sparcv9.tar.gz, is-x64.tar.Z, is-x64.tar.gz, ows-i586.exe, and ows-x64.exe. A modal dialog box is centered on the screen, titled 'You must accept the Oracle Technology Network License Agreement for Oracle Java SE to download this software.' It contains a checkbox with a green checkmark, followed by the text 'I reviewed and accept the Oracle Technology Network License Agreement for Oracle Java SE'. Below this, it says 'You will be redirected to the login screen in order to download the file.' At the bottom of the dialog is a green button labeled 'Download jdk-8u251-windows-x64.exe' with a download icon. The checkbox and the download button are highlighted with red circles and a red rectangle, respectively.

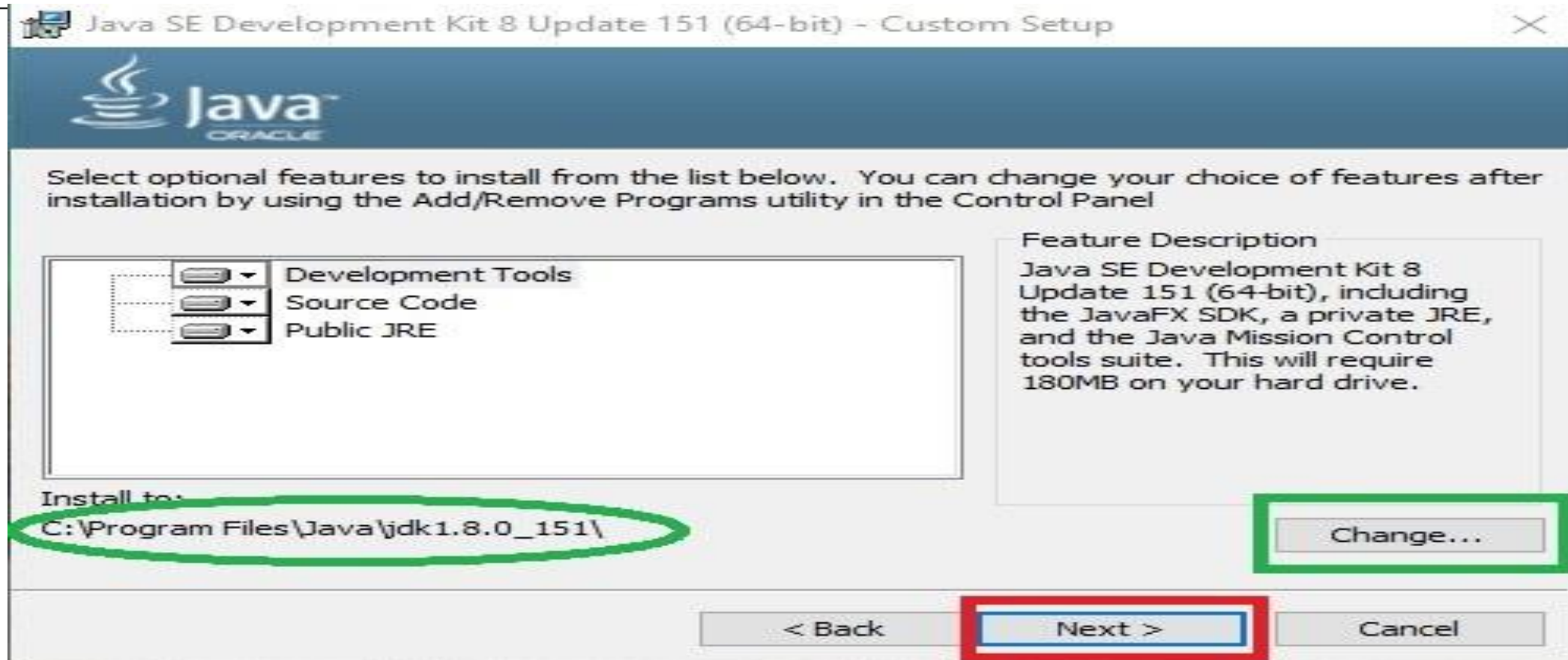
Step 4: We will now be navigated to **Oracle Login page**. We need to login to the account. As soon we log in, our download will start instantly as shown below:



Step 5: After the downloading procedure is complete, we need to run the installer. Once Java installation wizard opens, click on the **Next** button as shown below:



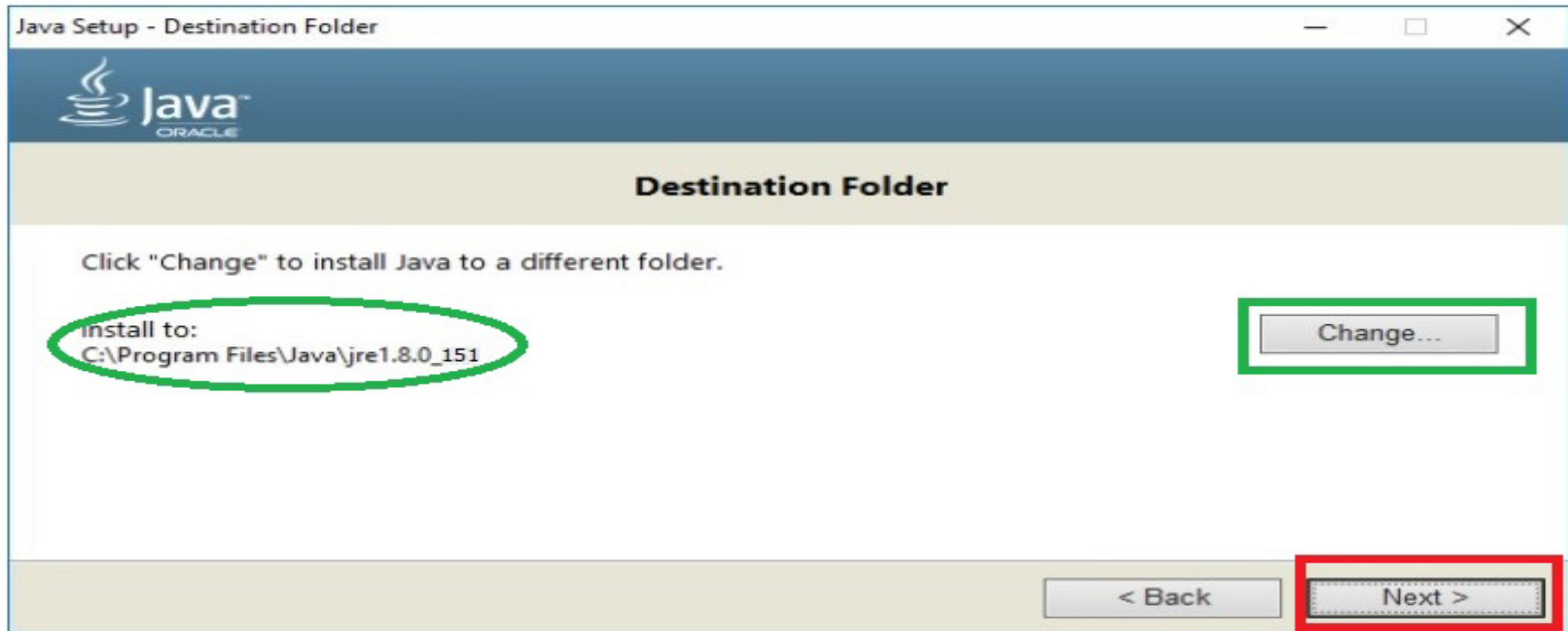
Step 6: Again click on the **Next** button if we wish to install Java development kit in the default directory(encircled with green color), or we can change this directory by clicking on **Change** button.



Step 7: The installation will begin as shown below:



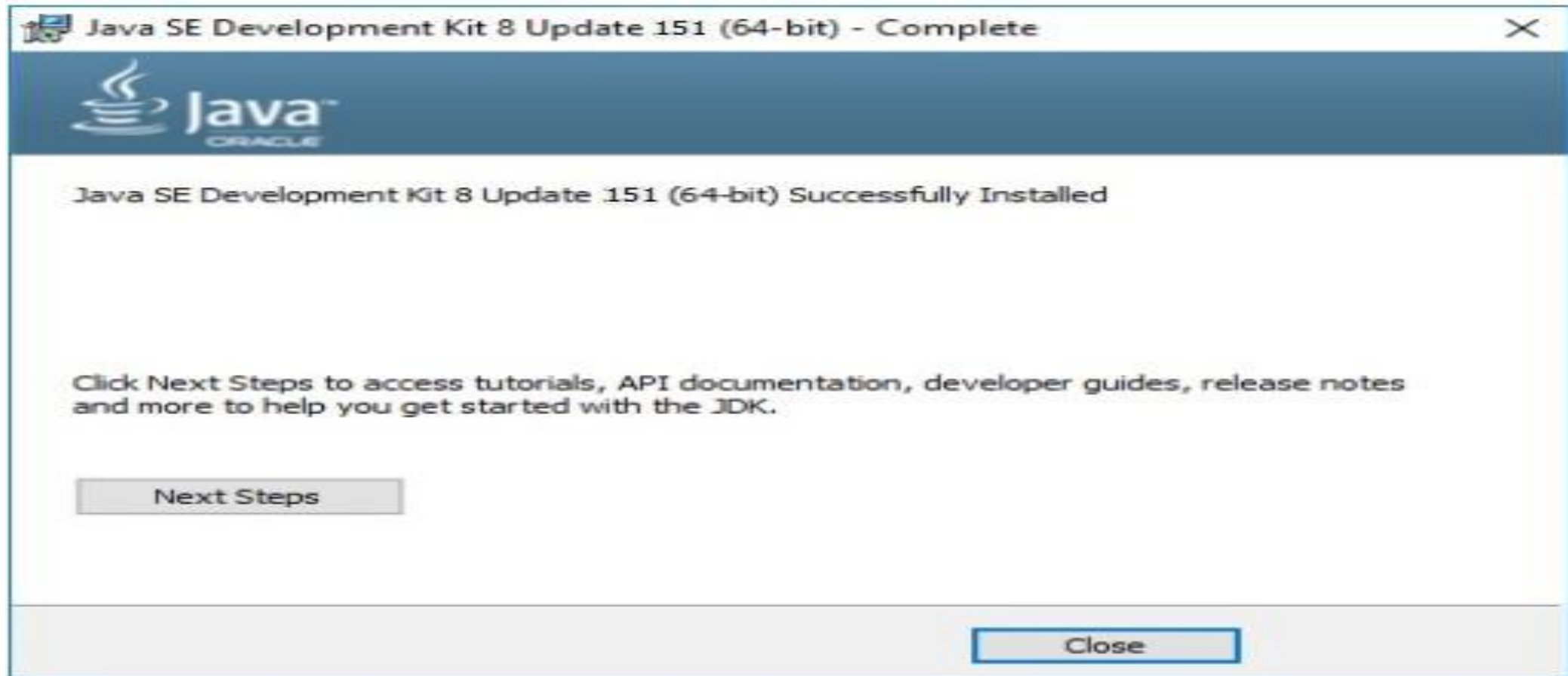
Step 8: Now, it will ask for the installation directory for **JRE(Java Runtime Environment)**. Again we can continue with the default directory or change it accordingly.



Step 9: The Java installation will be successfully completed as shown below:



Step 10: Finally, we can click on the **Close** button after the confirmation window appears that the Java is installed.



Simple Java Program

```
class myfirstjavapgm
{
    public static void main(String[] args)
    {
        System.out.println("First Java program.");
    }
}
```

Type the above program in notepad and save using the class name.java.This file is saved as MyFirstJavaProgram.java



Compiling and running a java program

- Compiling and running a java program is very easy after JDK installation. Following are the steps
- Open a command prompt window and go to the directory where you saved the java program (myfirstjavapgm.java). Assume it in Desktop:\dhilipjava.
- Next, set path for your jdk file. Ex: C:\Users\admin\Desktop\dhilipjava
- Type 'javac myfirstjavapgm.java' and press enter to compile your code. If there are no errors in your code, the command prompt will take you to the next line (Assumption: The path variable is set).
- Now, type ' java myfirstjavapgm ' to run your program.
- You will be able to see the result printed on the window.

Output Screen

```
C:\Users\admin\Desktop\dhilipjava>javac myfirstjavapgm.java  
  
C:\Users\admin\Desktop\dhilipjava>java myfirstjavapgm  
First Java program.
```



JAVA Working Environment

Three Components

- JDK-Java Development Kit
- JVM-Java Virtual Machine
- JRE-Java Runtime Environment



JDK

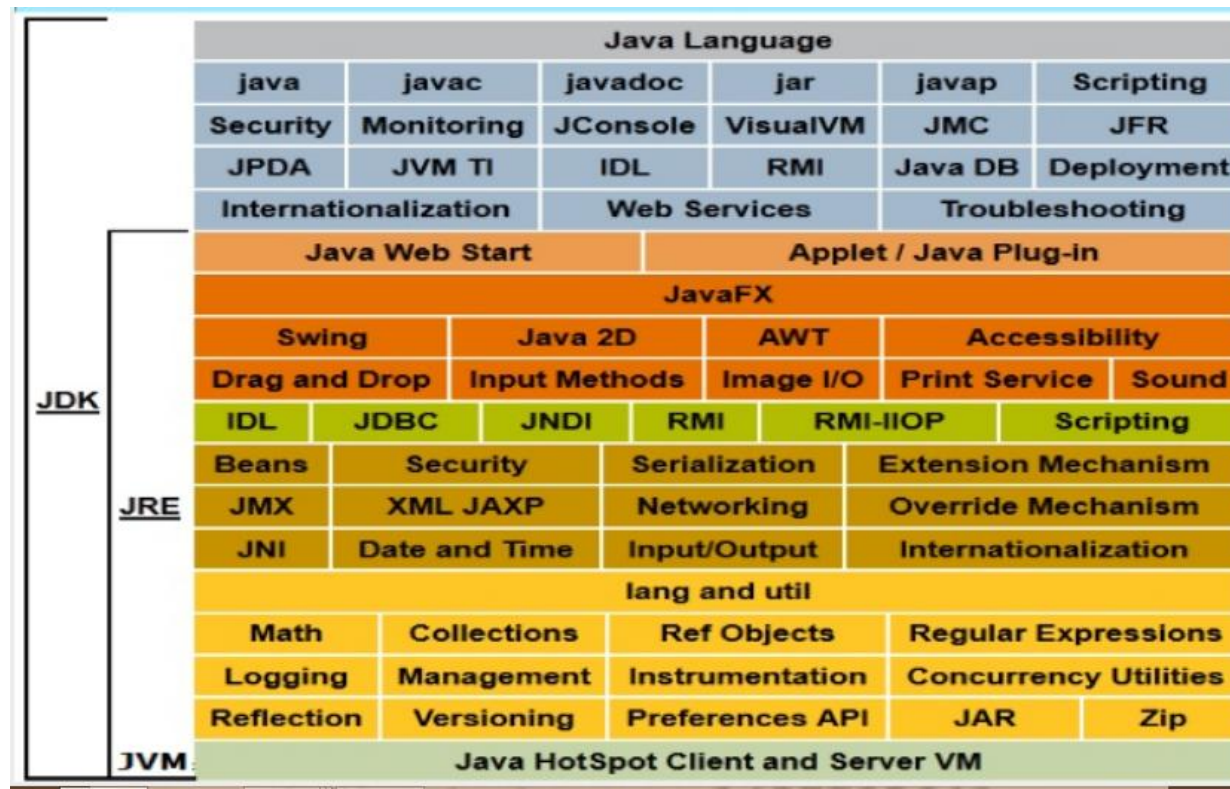
The Java Development Kit (JDK) is primary components.

It physically exists.

It is collection of programming tools and JRE, JVM.

Writing Java applets and applications need development tools like JDK.

JDK Architecture



Source:ictacademy.com



JRE

JRE is an acronym for Java Runtime Environment.

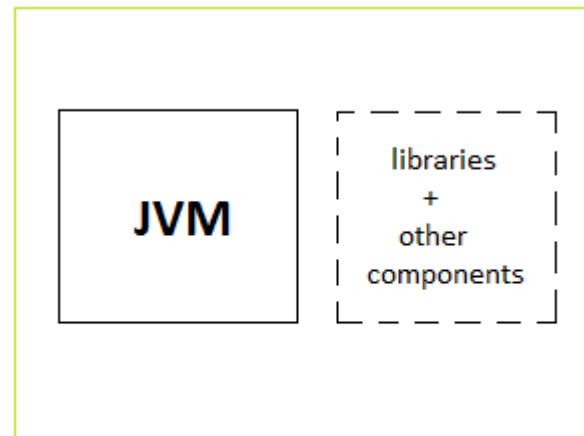
It is also written as Java RTE.

The Java Runtime Environment is a set of software tools.

It physically exists.

It contains a set of libraries + other files that JVM uses at runtime.

JRE Architecture



JRE - Java Runtime Environment

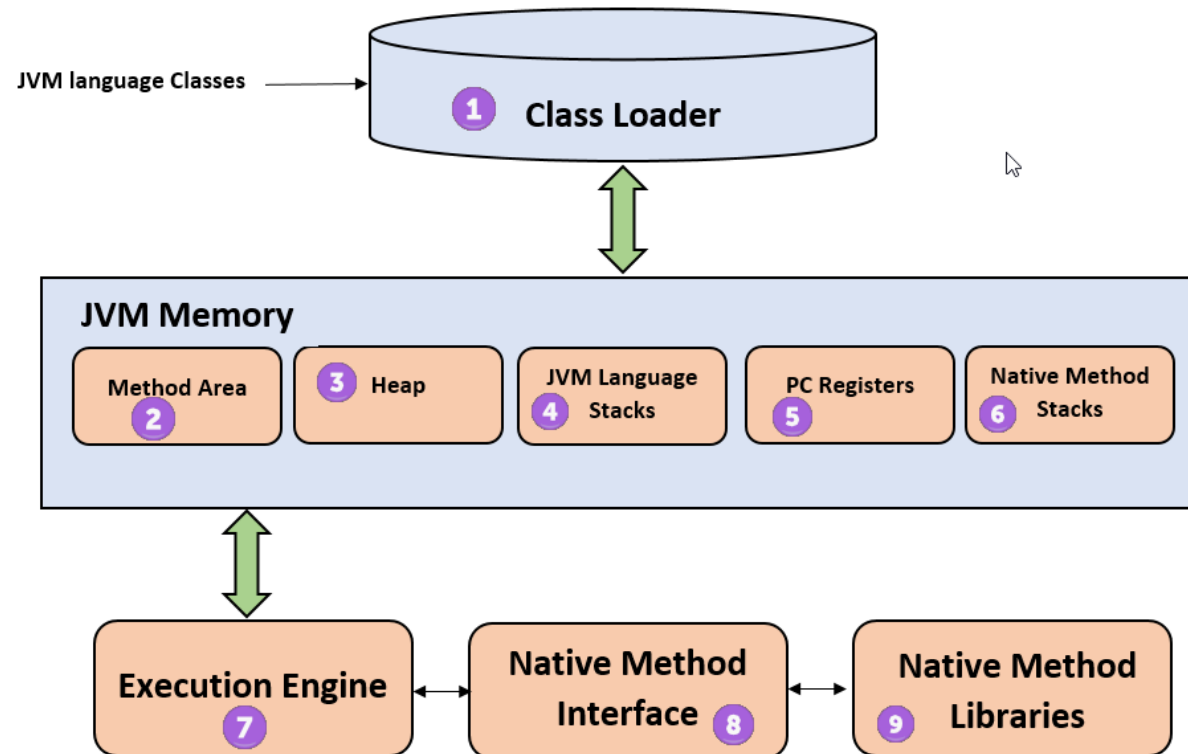
Source: studytonight.com



JVM

- JVM is a engine that provides runtime environment to drive the Java Code or Applications.
- It converts Java byte code into machine languages.
- Java compiler produces code for a virtual machine known as Java Virtual Machine.

JVM Architecture



Source:guru99.com



Uses of JDK,JRE,JVM

- The Java Virtual Machine is built right into your Java software download and helps run Java applications.
- The JRE allows applets written in the Java programming language to run inside various browsers.

Difference between JDK,JRE,JVM

JDK	JRE	JVM
It stands for Java Development Kit.	It stands for Java Runtime Environment	It stands for Java Virtual Machine.
It is the tool necessary to compile, document and package java programs	JRE refers to a runtime environment in which Java bytecode can be executed.	It is an abstract machine. It is a specification that provides a runtime environment in which Java bytecode can be executed.
It contains JRE+development tools.	It's an implementation of the JVM which physical exists.	JVM follow three notations: Specification, Implementation and Runtime Instance

Topic 1

OOP Concepts

Topic 2

Characteristics of JAVA

Topic 3

OOP in JAVA

Topic 4

JAVA Working Environment

First Lesson Summary

Learned about OOP concepts, Characteristics of JAVA, OOP in JAVA, JAVA Working Environment

Fundamental Programming Constructs

Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

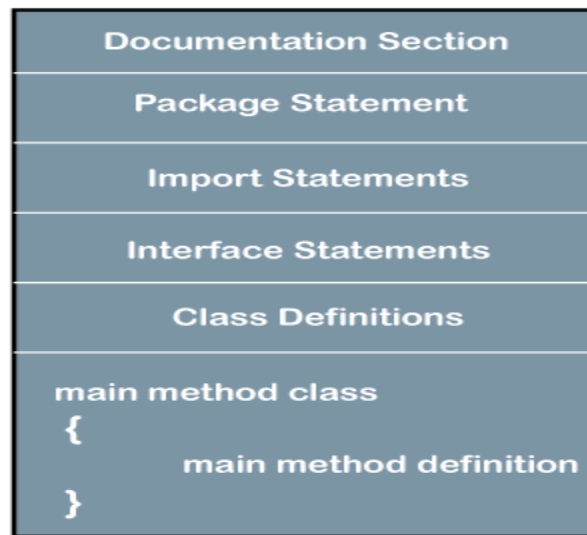
Lesson 4. Constructors

Lesson 5. Inheritance

Introduction



- A program is a sequence of instructions that a computer can execute to perform some task. There are two basic blocks of programming: data and instructions and to work with data, you need to understand variable and data types; to work with instructions, you need to understand control structures and subroutines (methods).



Structure of Java Program

Structure of Java Program

- Java is called as Platform independent, Object oriented, and secure programming language.
- Using the java programming language, we can develop a wide variety of applications. So we want to understand the basic structure of java program in detail.

package details	→	<code>import java.io.*</code>
<code>class className</code>	→	<code>class Sum</code>
{		
Data members;	→	<code>int a, b, c;</code>
user_defined method;	→	<code>void display();</code>
<code>public static void main(String args[])</code>		
{		
Block of Statements;	→	<code>System.out.println("Hello Java !");</code>
}		
}		



Structure of Java Program

- A package is a collection of classes, interfaces and sub-packages. A sub package contains collection of classes, interfaces and sub-sub packages etc. `java.lang.*`; package is imported by default and this package is known as default package.
- Class is keyword used for developing user defined data type and every java program must start with a concept of class. "ClassName" represent a java valid variable name treated as a name of the class each and every class name in java is treated as user-defined data type.
- Data member represents either instance or static they will be selected based on the name of the class.

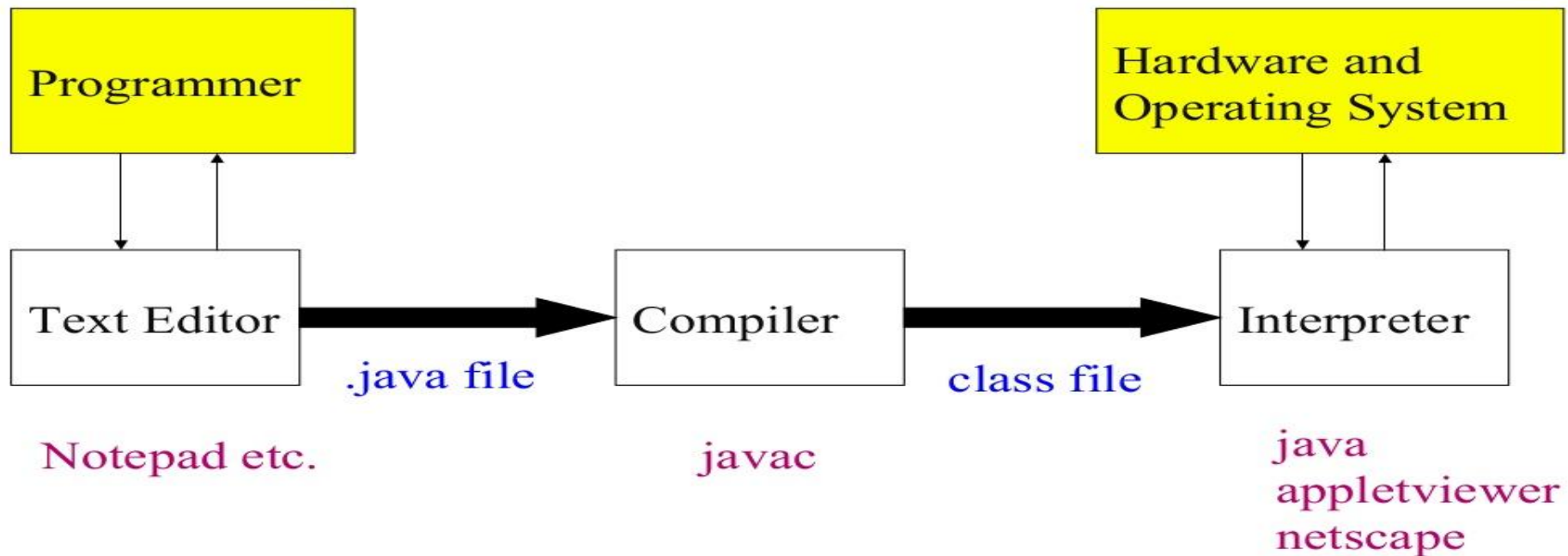
Structure of Java Program

- Each and every java program starts execution from the main() method. And hence main() method is known as program driver.
- Since main() method of java is not returning any value and hence its return type must be void.
- Since main() method of java executes only once throughout the java program execution and hence its nature must be static.
- Since main() method must be accessed by every java programmer and hence whose access specifier must be public.
- Each and every main() method of java must take array of objects of String.
- Block of statements represents set of executable statements which are in term calling user-defined methods are containing business-logic.
- The file naming convention in the java programming is that which-ever class is containing main() method, that class name must be given as a file name with an extension .java.

Structure of Java Program



Java is Compiled and Interpreted



Classes in Java



- . class defines a new data type.
- . Once defined, this new type can be used to create objects of that type.
- . Class is a template for an object, and an object is an instance of a class.
- . A class consists of Data members and methods.
- . The primary purpose of a class is to hold data/information. This is achieved with attributes which are also known as data members.
- . The member functions determine the behavior of the class, i.e. provide a definition for supporting various operations on data held in the form of an object.

Declaring Objects

- When you create a class, you are creating a new data type. You can use this type to declare objects of that type.
- Obtaining objects of a class is a two-step process. First, you must declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the new operator. The new operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, more or less, the address in memory of the object allocated by new. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated.

```
classname class -var = new classname ( );
```

Java Comments

- . Comments can be used to explain Java code, and to make it more readable. It can also be used to prevent execution when testing alternative code.
- . Single-line comments start with two forward slashes (//).
- . Any text between // and the end of the line is ignored by Java (will not be executed).
- . This example uses a single-line comment before a line of code:

```
// This is a comment  
System.out.println("Hello world")  
/* This is a Simple Java Program  
This is a comment */
```

Data Types



Characters

The char data type is used to store a **single** character. The character must be surrounded by single quotes, like 'A' or 'c'

```
public class Main {  
    public static void main(String[] args) {  
        char myGrade = 'B';  
        System.out.println(myGrade);  
    }  
}
```

Data Types



Strings

- The String data type is used to store a sequence of characters (text). String values must be surrounded by double quotes.

```
public class Main {  
    public static void main(String[] args) {  
        String greeting = "Hello World";  
        System.out.println(greeting);  
    }  
}
```


Variables

- Variables are symbolic names for memory storage.
- Like most compiled languages, variables must be declared before they can be used.
- Variables are a symbolic name given to a memory location.
- All variables have a type which is enforced by the compiler
- The general form of a variable declaration is:

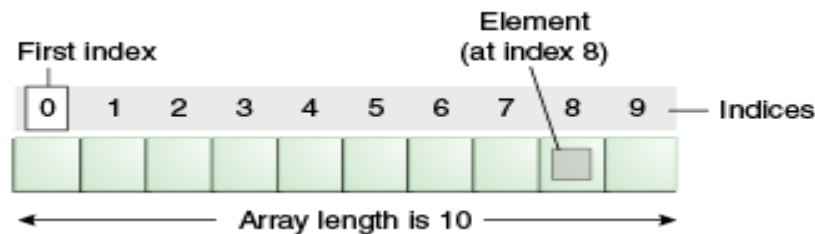
`type variable-name [= value][,variable-name[= value]];`

```
int total;  
float xValue  
= 0.0;  
boolean  
isFinished;
```

`String name;`

Arrays

An *array* is a container object that holds a fixed number of values of a single type. The length of an array is established when the array is created. After creation, its length is fixed. You have seen an example of arrays already, in the main method of the "Hello World!" application.



Each item in an array is called an *element*, and each element is accessed by its numerical *index*. As shown in the preceding illustration, numbering begins with 0. The 9th element, for example, would therefore be accessed at index 8.

Arrays – Example Program

```
class ArrayDemo {  
    public static void main(String[] args) {  
        // declares an array of integers  
  
        int[] anArray; // allocates memory for 10 integers anArray = new int[10]; // initialize first element anArray[0] = 100;  
        // initialize second element anArray[1] = 200; // and so forth  
  
        anArray[2] = 300;  
        anArray[3] = 400;  
        anArray[4] = 500;  
        anArray[5] = 600;  
        anArray[6] = 700;  
        anArray[7] = 800;  
        anArray[8] = 900;  
        anArray[9] = 1000;
```

```
        System.out.println("Element at index 0: " + anArray[0]);
```

Array Declaration & Initialization

The following code snippet shows the two dimensional array declaration in Java Programming Language:

```
Data_Type[][] Array_Name;
```

Data_type: It decides the type of elements it will accept. For example, If we want to store integer values, then the Data Type will be declared as int. If we want to store Float values, then the Data Type will be float.

Array_Name: This is the name to give it to this Java two dimensional array. For example, Car, students, age, marks, department, employees, etc.

```
for ( int i=0; i<n ;i++)  
{  
    for (int j=0; j<n; j++)  
    {  
        a[i][j] = 0;  
    }  
}
```

Array Declaration & Initialization

In the Java programming language, a multidimensional array is an array whose components are themselves arrays. This is unlike arrays in C or Fortran. A consequence of this is that the rows are allowed to vary in length, as shown in the following [MultiDimArrayDemo](#) program:

```
class MultiDimArrayDemo
{
    public static void main(String[] args)
    {
        String[][] names = { {"Mr. ", "Mrs. ", "Ms. "}, {"Smith", "Jones"} };
        // Mr. Smith
        System.out.println(names[0][0] + names[1][0]);
        // Ms. Jones
        System.out.println(names[0][2] + names[1][1]); } }
```

The output from this program is:

Mr. Smith Ms. Jones

Topic 1
Introduction - Structure of Java Program

Topic 2
Defining Class

Topic 2
Data Types

Topic 3
Variables

Topic 4
Operators

Topic 4
Control Flow

Topic 5
Arrays

Summary

LEARN FUNDAMENTAL PROGRAMMING
STRUCTURE FOR WRITING SIMPLE STAND ALONE
APPLICATION USING CONTROL, LOOPING AND
ARRAYS.

Access Specifiers



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental Programming Constructs

Lesson 3: Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance

Access Specifiers

There are two types of modifiers in Java:

- access modifiers
- non-access modifiers.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. The different types of access specifiers are as follows

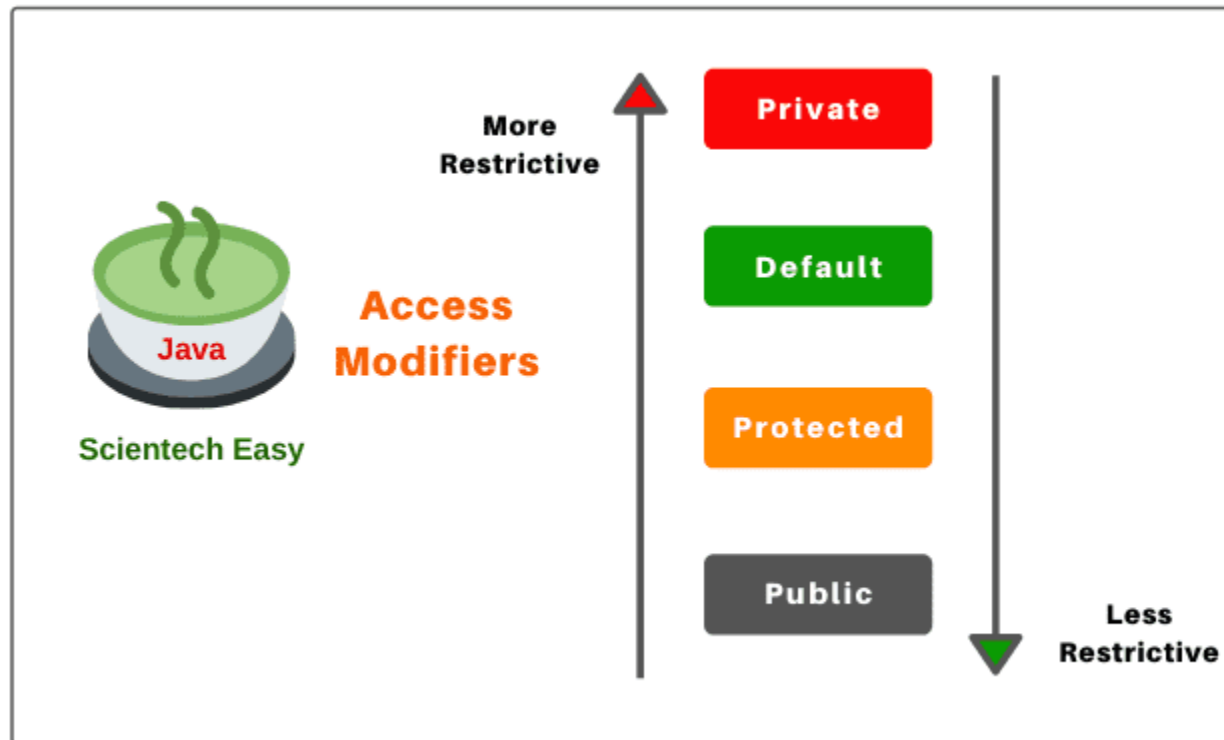
private: When a member of a class is specified as private, then that member can only be accessed by other members of its class.

public: When a member of a class is modified by public, then that member can be accessed by any other code.

protected: applies only when inheritance is involved.

default: same as specifying public.

Access Specifiers



Access Specifiers



	default	private	protected	public
Same Class	Yes	Yes	Yes	Yes
Same package subclass	Yes	No	Yes	Yes
Same package non-subclass	Yes	No	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Private



```
class A
{
    private int data=40;
    private void msg()
    {
        System.out.println("Hello java");
    }
}
public class Simple1
{
    public static void main(String args[])
    {
        A obj=new A();
        System.out.println(obj.data); //Compile Time Error
        obj.msg(); //Compile Time Error
    }
}
```

Output



```
C:\Users\admin\Desktop\dhilipjava>javac Simple1.java
Simple1.java:14: error: data has private access in A
        System.out.println(obj.data); //Compile Time Error
                              ^
Simple1.java:15: error: msg() has private access in A
        obj.msg(); //Compile Time Error
              ^
2 errors
```

Public



Public members of any class are accessible anywhere in the program inside the same class and outside of class, within the same package and outside of the package. The other name of public is universal access specifier

```
class Hello
{
    public int a=20;
    public void show()
    {
        System.out.println("Hello java");
    }
}
public class Demo
{
    public static void main(String args[])
    {
        Hello obj=new Hello();
        System.out.println(obj.a);
        obj.show();
    }
}
```

output



```
C:\Users\admin\Desktop\dhilipjava>javac Demo.java
```

```
C:\Users\admin\Desktop\dhilipjava>java Demo
```

```
20
```

```
Hello java
```

Protected:



Protected members of the class are accessible within the same class and another class of the same package and also accessible in inherited class of another package. Protected are also called derived level access modifiers.

```
package pack1;  
public class B1  
{  
protected void show()  
{  
System.out.println("Hello Java");  
}  
}
```

Protected member Access Example

```
public class Shape
{
    protected double height; // To hold height.    protected double width ; // To hold width or base    public void setValues( double height ,
    double width )
    {

    this. height = height;    this. width = width ;
    } }

public class Rectangle extends Shape
{
    public double getArea ()
    {
    return height * width ; // accessing protected members
    } }

public class TestProgram
{
    public static void main( String [] args)
    {

    Rectangle rectangle = new Rectangle ();    rectangle. setValues (5 ,4);
    System . out. println (" Area of rectangle : " + rectangle. getArea ());
    } }
```


Static Members



- . Static members are those which belongs to the class and you can access these members without instantiating the class.
- . The static keyword can be used with methods, fields, classes (inner/nested), blocks.
- . **Static Methods** – You can create a static method by using the keyword *static*. Static methods can access only static fields, methods. To access static methods there is no need to instantiate the class, you can do it just using the class name as

Static Member



```
public class MyClass
{
    public static void sample()
    {
        System.out.println("Hello");
    } public static
    void main(String args[])
    {
        MyClass.sample();
    }
}
```

Topic 1
Access Specifiers

Topic 2
Static Members

Third Lesson Summary

From this topic we learned the visibility about the classes and data members.

Constructors



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance

Constructors



In Java, a constructor is a block of codes similar to the method.

It is called when an instance of the class is created.

At the time of calling constructor, memory for the object is allocated in the memory.

It is a special type of method which is used to initialize the object.

Types of Constructors



A constructor having no parameter is known as default constructor and no-arg constructor.

Constructor having parameter is known as parameterized constructor.

```
class Box {  
    double    width ;  
    double    height;  
    double    depth ;  
  
    // default Constructor  
    Box ()  
    {        width =  3;        height =  4;        depth = 2;        }  
    // parameterized Constructor  
    Box( double w, double h, double d)  
    {        width = w;        height = h;        depth = d;        }  
}
```

Example

```
class BoxDemo7 {  
    public static void main( String args [])  
    {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box(3, 6, 9);  
        double vol;  
        // get volume of first box  
        vol = mybox1 . volume ();   System . out .  
        println("Volume is " + vol);  
        // get volume of second box  
        vol = mybox2 . volume ();   System . out .  
        println("Volume is " + vol);  
    }  
}
```

this keyword

this can be used inside any method to refer to the current object. That is, this is always a reference to the object on which the method was invoked.

```
Box( double w, double h, double d) {  
    this.width = w;  
    this.height = h;  
    this.depth = d;  
}
```


Stack Implementation

```
class Stack
{
    int stck [] = new int [10];
    int tos;
    Stack ()
    {
        tos = -1;
    }
    void push(int item)
    {
        if (tos == 9)
            System.out.println("Stack is full.");
        else
            stck[++tos] = item;
    }
    int pop ()
    {
        if (tos < 0) {
            System.out.println("underflow");
            Stack return 0;
        }
        else
            return stck[tos--];
    }
}
```

```
class TestStack {  
    public static void main( String args []) {  
        Stack mystack1 =new Stack();  
        Stack mystack2 = new Stack();  
        for( int i=0; i <10; i++) mystack1 . push(i);  
        for(int i=10; i <20; i++) mystack2 . push(i);  
        System . out . println("Stack in mystack1:");  
        for( int i=0; i <10; i++)  
            System . out . println( mystack1 . pop());  
        System . out . println("Stack in mystack2  
        :");  for( int i=0; i <10; i++)  
            System . out . println( mystack2 . pop());  
    }  
}
```

Topic 1
Default Constructor

Topic 2
Parameterized Constructor

Third Lesson Summary

Learnt the uses of constructors to assign initial values to the data members through default and parameterized constructors

Inheritance



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5. Inheritance



Inheritance Introduction

Definition of Inheritance

Inheritance is the mechanism of deriving new class from old one, old class is known as superclass and new class is known as subclass.

The subclass inherits all of its instances variables and methods defined by the superclass and it also adds its own unique elements.

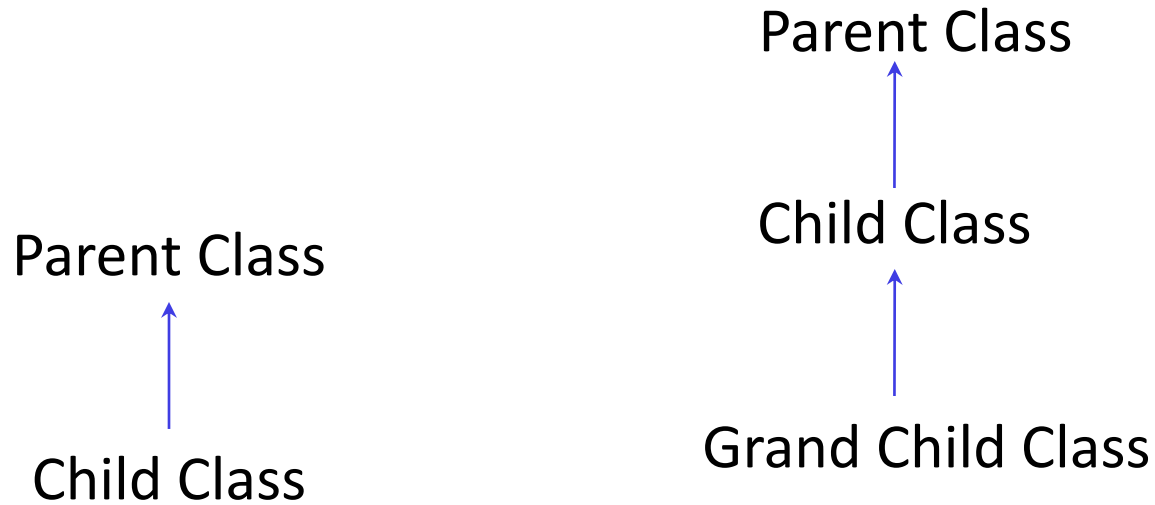
Thus, we can say that subclass is specialized version of superclass.

Syntax

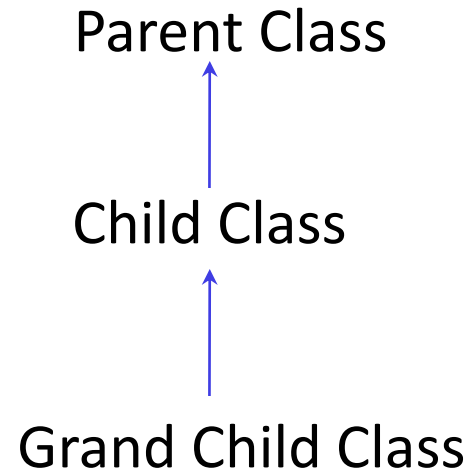
extends is the keyword used to inherit the properties of a class

```
class Super
{ .....
..... }
class Sub extends Super
{ .....
..... }
```

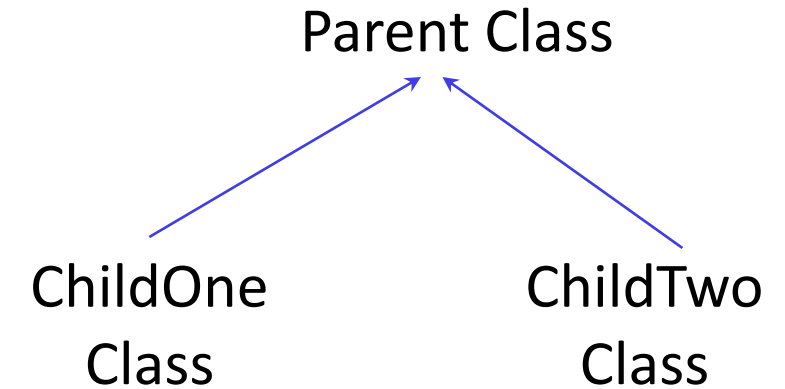
Types of Inheritance



**Single
Inheritance**



**Multilevel
Inheritance**



**Hierarchical
Inheritance**

Single Inheritance

- A class inherits another class
- For example the Dog class inherits the Animal class as shown below

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class TestInheritance{  
public static void main(String args[]){  
Dog d=new Dog();  
d.bark();  
d.eat();  
}}
```

Output
barking...
eating...



Multiple Inheritance

- chain of inheritance
- its like child, parent and grandparent relationship
- for example a BabyDog class inherits the Dog class which again inherits the Animal class

Example Program

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class BabyDog extends Dog{  
void weep(){System.out.println("weeping...");}  
}  
class TestInheritance2{  
public static void main(String args[]){  
BabyDog d=new BabyDog();  
d.weep();  
d.bark();  
d.eat();  
}}}
```

Output
weeping...
barking...
eating...

Hierarchical Inheritance

- when two or more classes inherits a single class
- for example Dog class and Cat class inherits the Animal class

Example Program

```
class Animal{  
void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}  
class Cat extends Animal{  
void meow(){System.out.println("meowing...");}  
}  
class TestInheritance3{  
public static void main(String args[]){  
Cat c=new Cat();  
c.meow();  
c.eat();  
//c.bark();//Compile Time Error  
}}
```

Output
meowing...
eating...



INTERFACE

- An **interface in java** is a blueprint of a class.
- It has static constants and abstract methods only.
- The interface in java is a **mechanism to achieve fully abstraction**.
- There can be only abstract methods in the java interface not method body.
- It is used to achieve fully abstraction and multiple inheritance in Java.

Program on Interface

```
interface IntStack
{
    void push(int item);
    int pop();
}
```

```
class FixedStack implements IntStack
{
    private int stack[];
    private int tos;
    FixedStack(int size)
    {
        stack=new int[size];
        tos=-1;
    }
    public void push(int item)
    {
        if(tos==stack.length-1)
            System.out.println("stack is full");
        else
            stack[++tos]=item;
    }
    public int pop()
    {
        if(tos<0)
        {
            System.out.println("stack underflow");
            return 0;
        }
    }
}
```

```
    }
    else
        return stack[tos--];
}
public static void main(String args[])
{
    FixedStack mystack1=new FixedStack(5);
    FixedStack mystack2=new FixedStack(8);
    for(int i=0;i<6;i++) mystack1.push(i);
    for(int i=0;i<9;i++) mystack2.push(i);
    System.out.println("stack in mystack1");
    for(int i=0;i<6;i++)
        System.out.println(mystack1.pop());
    System.out.println("stack in mystack2");
    for(int i=0;i<9;i++)
        System.out.println(mystack2.pop());
}
```

Output

```
C:\Users\admin\Desktop\dhilipjava>java FixedStack
stack is full
stack is full
stack in mystack1
4
3
2
1
0
stack underflow
stack in mystack2
7
6
5
4
3
2
1
0
stack underflow
0
```

Topic 1

Inheritance Introduction

Topic 2

Types of Inheritance

Topic 3

Private Member Access

Topic 4

Usage of Super

Topic 5

Protected Member Access

Topic 6

When constructors are called

Lesson Summary

Understood Inheritance and solve real world problem by implementing different types of inheritance

Packages



Lesson 1. Object Oriented programming Concepts

Lesson 2. Fundamental programming Constructs

Lesson 3. Access Specifiers

Lesson 4. Constructors

Lesson 5 : Inheritance

Lesson 6. Packages

Packages in Java



A **java package** is a group of similar types of classes, interfaces and sub-packages.

Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Here, we will have the detailed learning of creating and using user-defined packages.



Packages in Java

Java uses file system directories to store packages. For example, the **.class** files for any classes you declare to be part of **MyPackage** must be stored in a directory called **MyPackage**.

You can create a hierarchy of packages. The general form of a multileveled package statement is shown here: **package pkg1[.pkg2[.pkg3]];**

Example

```
Simple.java - Notepad
File Edit Format View Help

package mypack;
public class Simple{
    public static void main(String args[]){
        System.out.println("Welcome to
package");
    }
}
```

```
Command Prompt
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\guru>d:
D:\>cd java
D:\Java>set path="C:\Program Files\Java\jdk1.8.0_261\bin"
D:\Java>javac -d . Simple.java
D:\Java>java mypack.Simple
Welcome to package
D:\Java>
```

How to access package from another package?



There are three ways to access the package from outside the package.

- `import package.*;`
- `import package.classname;`
- fully qualified name.

Using packagename.*



Using packagename.*

If you use `package.*` then all the classes and interfaces of this package will be accessible but not sub packages. The `import` keyword is used to make the classes and interface of another package accessible to the current package.

```
package pack;
public class A
{
    public void msg()
    {
        System.out.println("Hello this is using packagename.*");
    }
}
```

```
package mypack;
import pack.*;
class B
{
    public static void main(String args[])
    {
        A obj = new A();
        obj.msg();
    }
}
```

Using packagename.classname



Using packagename.classname

If you import `package.classname` then only declared class of this package will be accessible.

Sample package created

 mypack	5/21/2020 12:19 PM	File folder
 mypack1	5/21/2020 12:24 PM	File folder
 mypack2	5/21/2020 12:33 PM	File folder
 p1	9/11/2020 10:00 AM	File folder
 pack	5/21/2020 12:19 PM	File folder
 pack1	5/21/2020 12:37 PM	File folder

Using fully qualified name



If you use fully qualified name then only declared class of this package will be accessible.

Now there is no need to import.

But you need to use fully qualified name every time when you are accessing the class or interface.

Topic 1

Packages in Java

Topic 2

Importing Packages

Topic 3

Builtin Packages in Java

Sixth Lesson Summary

Learned about Packages in Java, importing Packages, Builtin Packages in Java, JavaDoc, JavaDoc tags

Thank You!